

第2章 微处理器

- 2.1 8086/8088微处理器
- 2.2 案例实现

目标案例2.1：8086 CPU最小系统设计

要求：设计8086 CPU最小工作模式下的系统配置，并给出仿真实现。本章节后面提到的内存储器及接口均是在CPU最小系统基础上工作的。

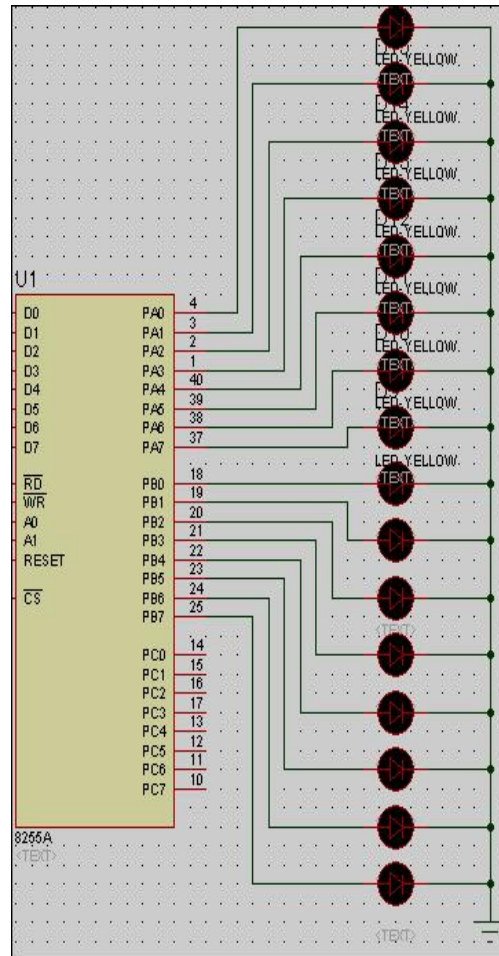
目标案例2.2：8086 CPU在流水灯控制系统中的应用

要求：在8086最小工作模式下，对流水灯进行控制。

- 当开关K0闭合（K0=0，接地），8086 CPU使流水灯全部熄灭；
- 当开关K0断开时（K0=1，接入高电平），8086 CPU使流水灯依次点亮。

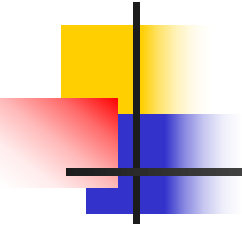
给出流水灯控制系统的原理图及控制程序，并仿真实现其功能。

流水灯



Intel CPU发展简史

微处理器型号	微处理器型号: 字长, 主频, 晶体管个数, 制作工艺
1971 年 4004	4 位,108KHz,2300,10 μ m,第一块个人微处理器, 每秒 5 万条指令, 计算器
1972 年 8008	8 位,500-800KHz,3500,10 μ m,获得了 <u>Datapoint</u> 公司开发的指令集
1974 年 8080	8 位,2MHz, 4500, 6 μ m,每秒 29 万条指令
1978 年 8086-8088	16 位,5MHz,29000,3 μ m,地址线 20 条,
1982 年 80286	16 位,6MHz,134000,1.5 μ m
1985 年 80386	32 位,16MHz,275000,1.5 μ m,每秒 500 万条指令,多任务
1989 年 80486	32 位,25MHz,1200000,1 μ m,内建的数学协处理器
1993 年 Pentium (586)	32 位,66MHz,320 万,0.8 μ m,集成语音、声音、手写体、图片等数据
1997-Pentium II	32 位,300MHz,750 万,0.25 μ m; Pentium II <u>Celeron</u> :带有二级缓存
1999-Pentium III	32 位,500MHz,950 万,0.18 μ m,
2000-Pentium 4	32 位,1.5GHz,4200 万,0.18 μ m,多个型号
2002-2004 超线程 P4	32 位,3.06Hz,5500 万,0.13 μ m,每秒 30 亿周期
2005-2006 双核处理器	64 位,3.5GHz
2006 酷睿 2 双核	64 位,2.91 亿,
2011 新的处理器产品架构	64 位,i3:双核,2MB 二级缓存; i5:四核,4MB 二级缓存; i7:四核八线程/六核十二线
2014 i7-5960X	64 位,22nm,3.5GHz,20MB 二级缓存,8 核 16 线程
2015 第 5 代 Core 系列	64 位,14nm



国产处理器简介

基于**ARMv8架构**--飞腾CPU、鲲鹏CPU（华为）；
基于**X86架构** -- 兆芯（个人PC）、海光CPU（服务器）；
100%自主可控--龙芯、申威

龙芯中科

- **2001年5月**

在中科院计算所知识创新工程的支持下，龙芯课题组正式成立

- **2001年8月19日**

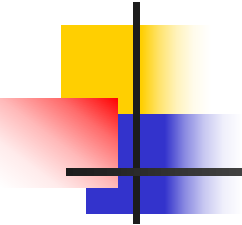
龙芯**1号**设计与验证系统成功启动**Linux**操作系统

- **2002年8月10日**

我国首款通用**CPU**龙芯**1号**（代号**X1A50**）流片成功

- **2023年8月**

基于龙架构的新一代四核处理器龙芯 **3A6000** 已经成功流片。这是我国自主桌面 **CPU** 设计领域的的一项重大里程碑成果。根据综合测试结果，龙芯 **3A6000** 处理器的总体性能与英特尔公司 **2020** 年上市的第 **10** 代酷睿四核处理器相当，这表明我们在处理器性能方面已经取得了长足的进步。



2.1 8086/8088微处理器

2.1.1 8086CPU的内部结构

2.1.2 8086CPU的内部寄存器

2.1.3 8086CPU的工作模式和引脚信号

2.1.4 8086的内存储器器和I/O端口组织

2.1.5 8086最大模式系统和最小模式系统

2.1.6 8086CPU的工作时序

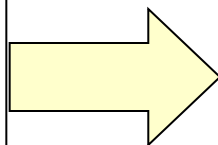
2.1.1 8086CPU的内部结构



型号	字长	地址总线AB	数据总线DB
8086	16位	20位 (1MB)	16位
8088	准16位	20位 (1MB)	8位

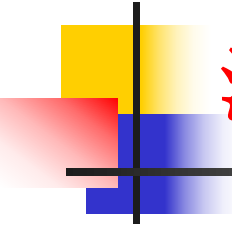
采用流水线技术

- (1) 取指令
- (2) 分析指令
- (3) [取操作数]
- (4) 执行指令
- (5) [保存结果]



EU: 分析指令 +
执行指令

BIU: 取指令 +
取操作数 +
存结果(总线操作)

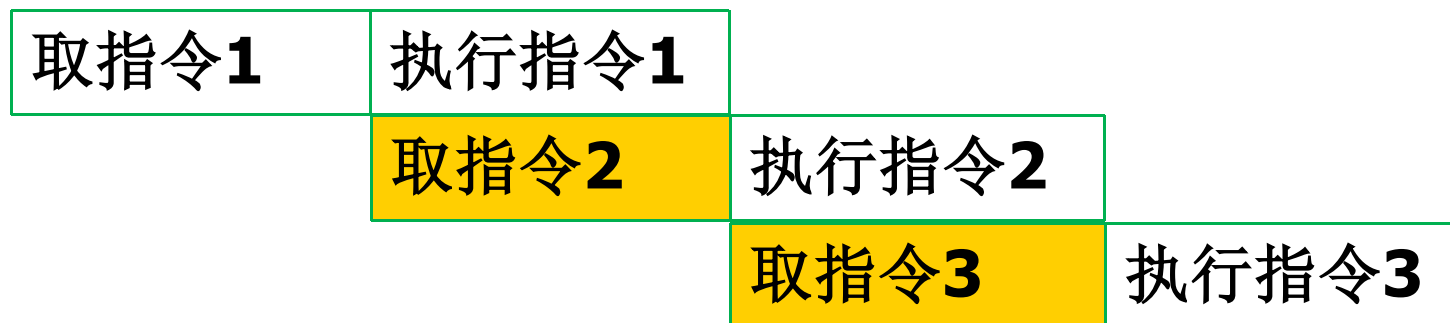


流水线和并行处理:

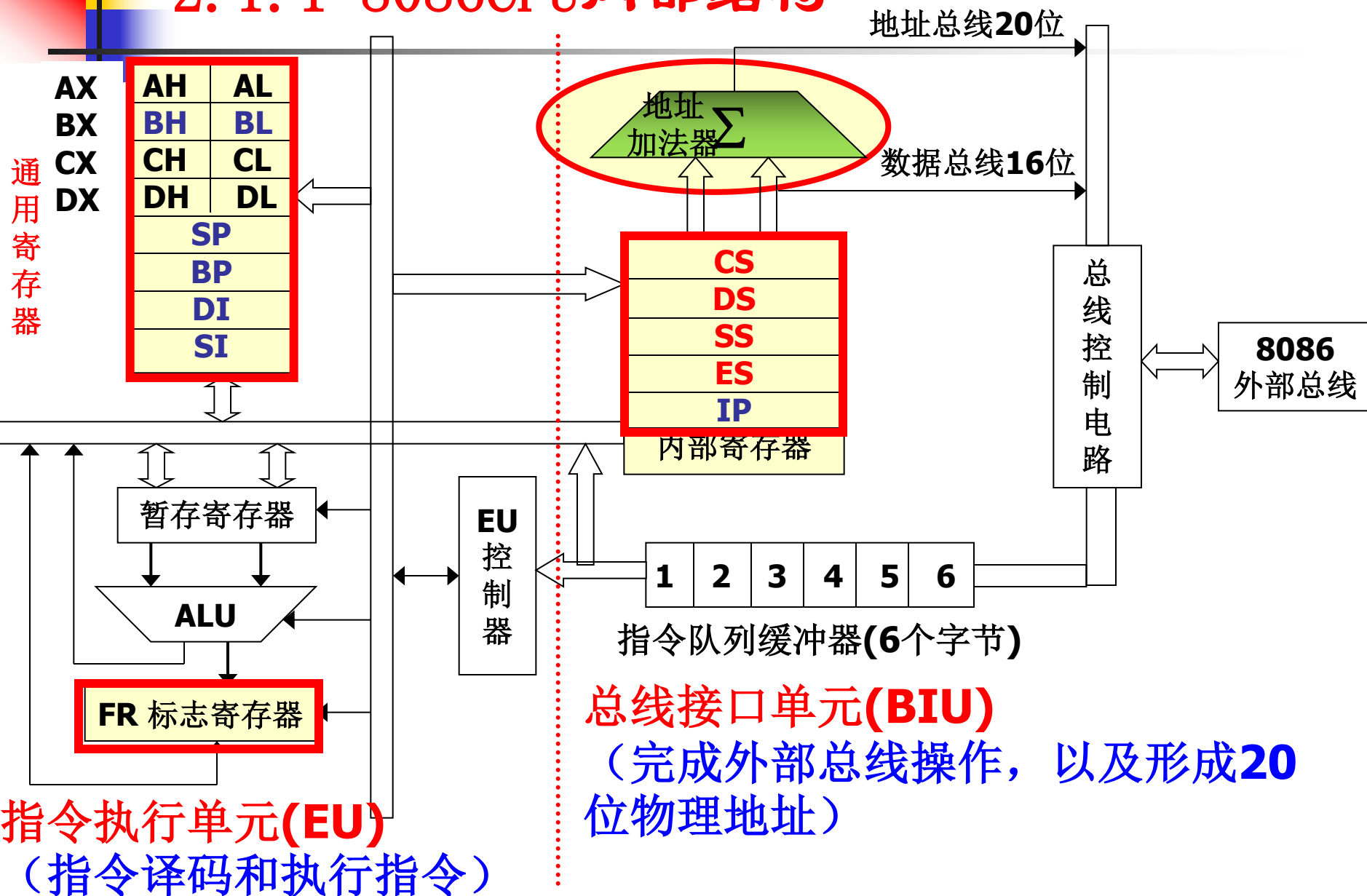
非流水线操作:



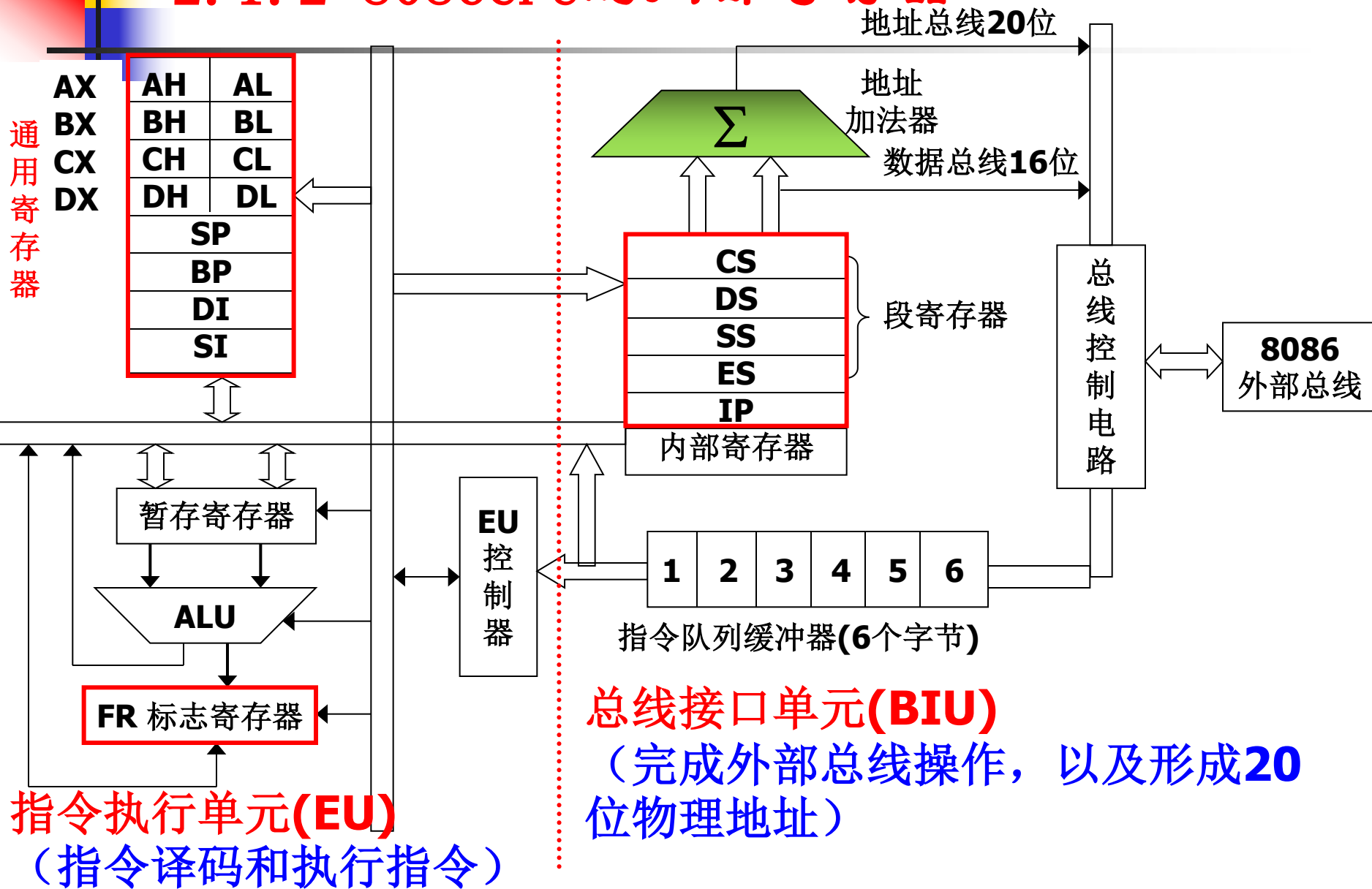
流水线操作:



2.1.1 8086CPU内部结构



2.1.2 8086CPU的内部寄存器



2.1.2 8086CPU的内部寄存器

	15	8 7	0		
AX	AH		AL	累加器	通用数据寄存器
BX	BH		BL	基址寄存器	
CX	CH		CL	计数器	
DX	DH		DL	数据寄存器	
	15		0		
	SP			堆栈指针	指针和变址寄存器
	BP			基址指针	
	SI			源变址	
	DI			目的变址	
	15		0		
	IP			指令指针	控制寄存器
	FR			标志寄存器	
	15		0		
	CS			代码段	段寄存器
	DS			数据段	
	SS			堆栈段	
	ES			附加段	

2.1.2 8086CPU的内部寄存器

1. 通用寄存器: 存放操作数、操作数地址及结果。

15 14.....1 0

AX
BX
CX
DX

AH	AL
BH	BL
CH	CL
DH	DL
SP	
BP	
SI	
DI	

累加器

基址寄存器

计数寄存器

数据寄存器

堆栈指针

基址指针

源变址寄存器

目的变址寄存器

常用寄存器

指针寄存器

变址寄存器

```
MOV  AL, 1      ; AL ← 01H
MOV  AX, 1      ; AX ← 0001H
```

2.1.2 8086CPU的内部寄存器

2. 专用寄存器

15 14-----1 0

CS	代码段寄存器
DS	数据段寄存器
ES	附加段寄存器
SS	堆栈段寄存器
IP	指令指针寄存器
FR	标志寄存器

地址	内存
段地址1: 0000H CS IP	代码段 (CS段)
段地址2: 0000H DS	数据段 (DS段)
段地址3: 0000H ES	附加段 (ES段)
段地址4: 0000H SS	堆栈段 (SS段)

- ◆ IP存放将要执行指令的偏移地址;
- ◆ CPU根据CS:IP在内存中取出指令代码的一个字节后, IP自动加1

2.1.2 8086CPU的内部寄存器

3. 标志寄存器FR（程序状态字PSW）

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
×	×	×	×	OF	DF	IF	TF	SF	ZF	×	AF	×	PF	×	CF

控制标志(**DF IF TF**): 通过指令设置

- **DF**----方向标志位，用于控制字符串指令地址变化方向
由指令 **STD**（减址）和**CLD**（增址）控制；
- **IF**-----中断允许标志位
由指令**STI**（允许）和**CLI**（禁止）控制；
- **TF**----单步标志位，**TF**置**1**，**CPU**单步执行指令；

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
×	×	×	×	OF	DF	IF	TF	SF	ZF	×	AF	×	PF	×	CF

状态标志：指令执行后自动设置

•CF-进位/借位标志

$\left\{ \begin{array}{l} \mathbf{CF=1} \text{ 有进位/借位} \\ \mathbf{CF=0} \text{ 无进位/借位} \end{array} \right.$

•ZF-零标志

$\left\{ \begin{array}{l} \mathbf{ZF=1} \text{ 结果为0} \\ \mathbf{ZF=0} \text{ 结果非0} \end{array} \right.$

•PF-奇偶标志

$\left\{ \begin{array}{l} \mathbf{PF=1} \text{ 低8位偶数个1} \\ \mathbf{PF=0} \text{ 低8位奇数个1} \end{array} \right.$

•SF-符号标志

$\left\{ \begin{array}{l} \mathbf{SF=1} \text{ 结果为负} \\ \mathbf{SF=0} \text{ 结果为正} \end{array} \right.$

•AF-辅助进位/借位标志

$\left\{ \begin{array}{l} \mathbf{AF=1} \text{ 低4位有进位/借位} \\ \mathbf{AF=0} \text{ 低4位无进位/借位} \end{array} \right.$

•OF-溢出标志

$\left\{ \begin{array}{l} \mathbf{OF=1} \text{ 溢出} \\ \mathbf{OF=0} \text{ 无溢出} \end{array} \right.$

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
×	×	×	×	OF	DF	IF	TF	SF	ZF	×	AF	×	PF	×	CF

例：

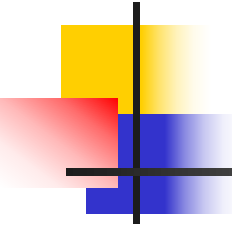
MOV AX, 9AE0H

ADD AX, 7080H

$$\begin{array}{r}
 1001 \ 1010 \ 1110 \ 0000 \\
 + \ 0111 \ 0000 \ 1000 \ 0000 \\
 \hline
 1 \ 0000 \ 1011 \ 0110 \ 0000
 \end{array}$$

结果：AX=0B60H; CF=1; PF=1; AF=0;

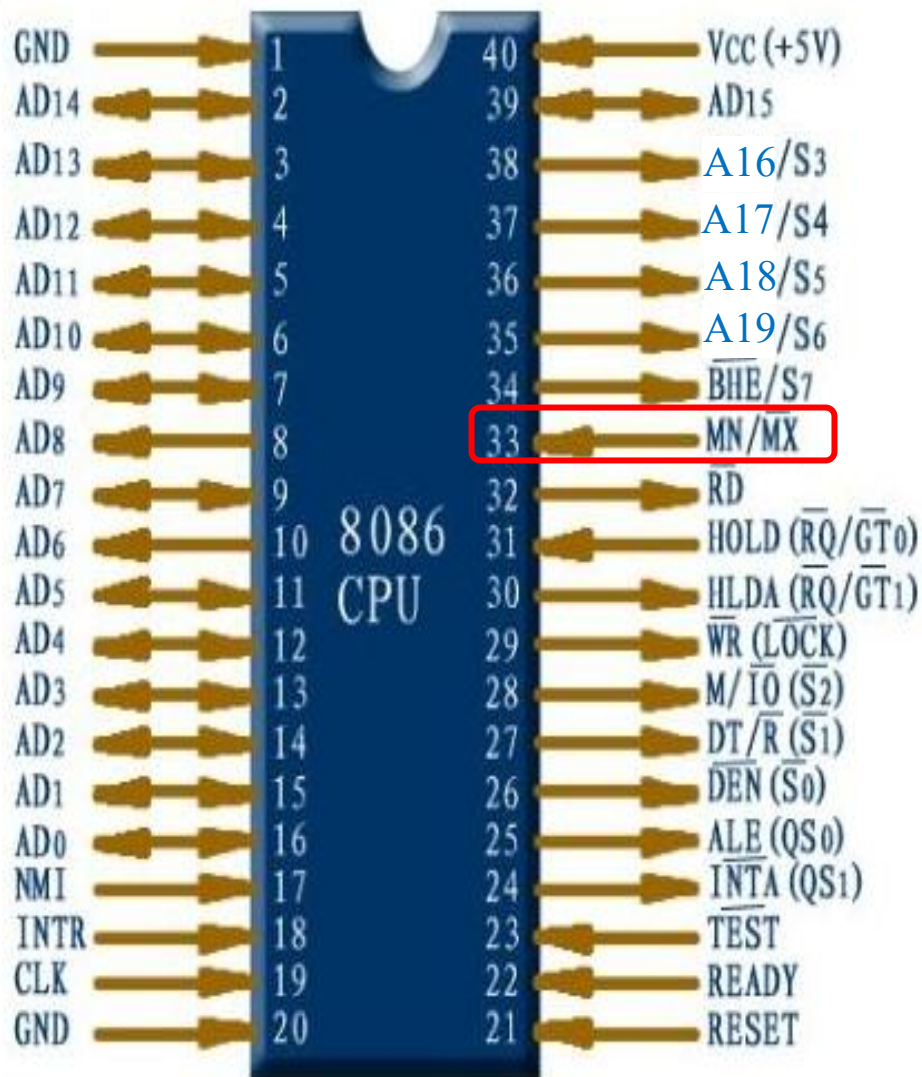
ZF=0; SF=0; OF=0



DEBUG 调试环境标志位的符号表示

标志位名称	标志位为 1	标志位为 0
OF 溢出（有/无）	OV	NV
DF 方向（减址/增址）	DN	UP
IF 中断（允许/关闭	EI	DI
SF 符号（负/正）	NG	PL
ZF 零（是/否）	ZR	NZ
AF 辅助进位（有/无）	AC	ZA
PF 奇偶（偶/奇）	PE	PO
CF 进位（有/无）	CY	NC

2.1.3 8086CPU的工作模式和引脚信号



40根引脚，双列直插封装结构
两种工作模式

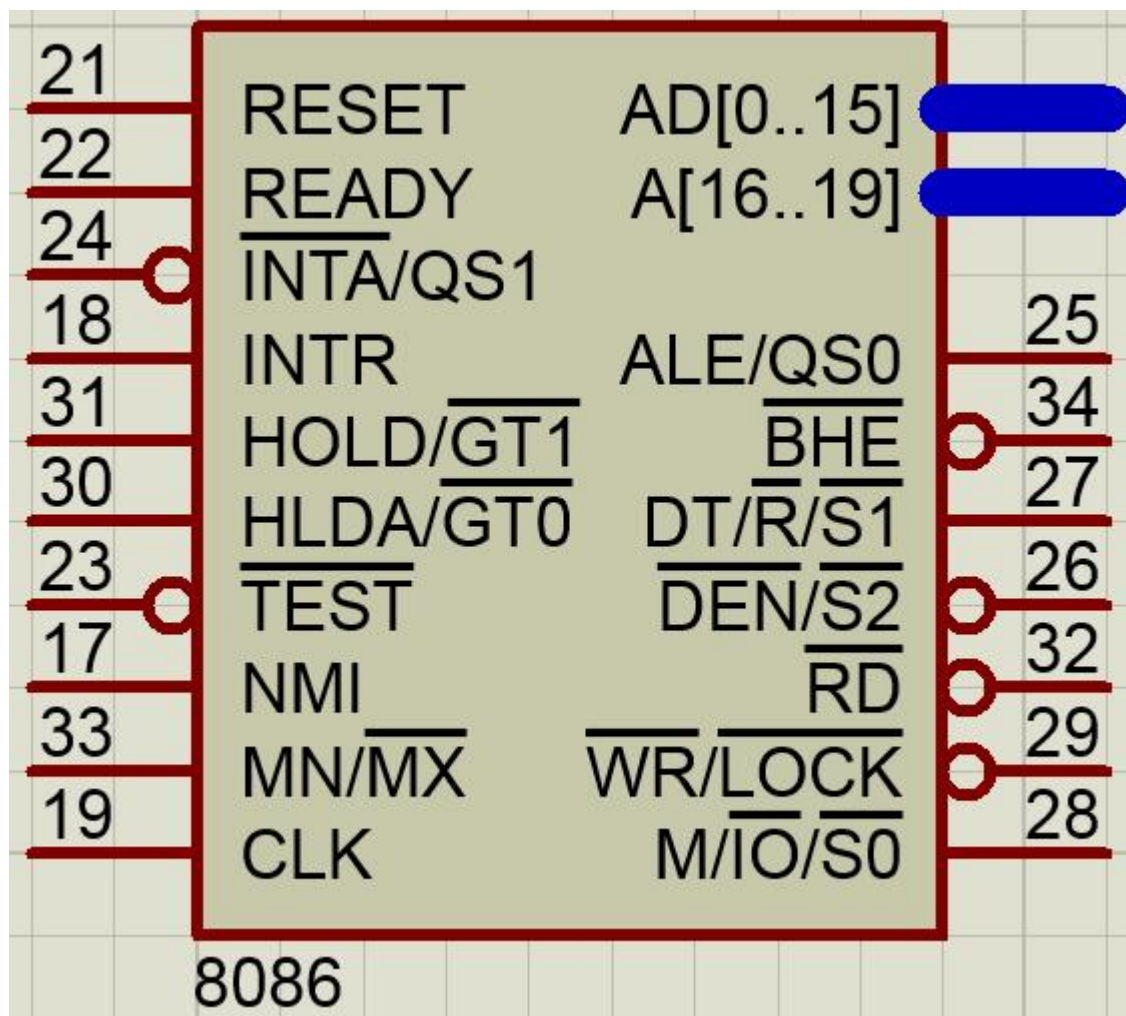
- **最小工作模式 (MN/MX*=1)**

系统中**只有一个8086微处理器**，所有的总线控制信号都由8086CPU产生，系统中的总线电路被减到最少

- **最大工作模式 (MN/MX*=0)**

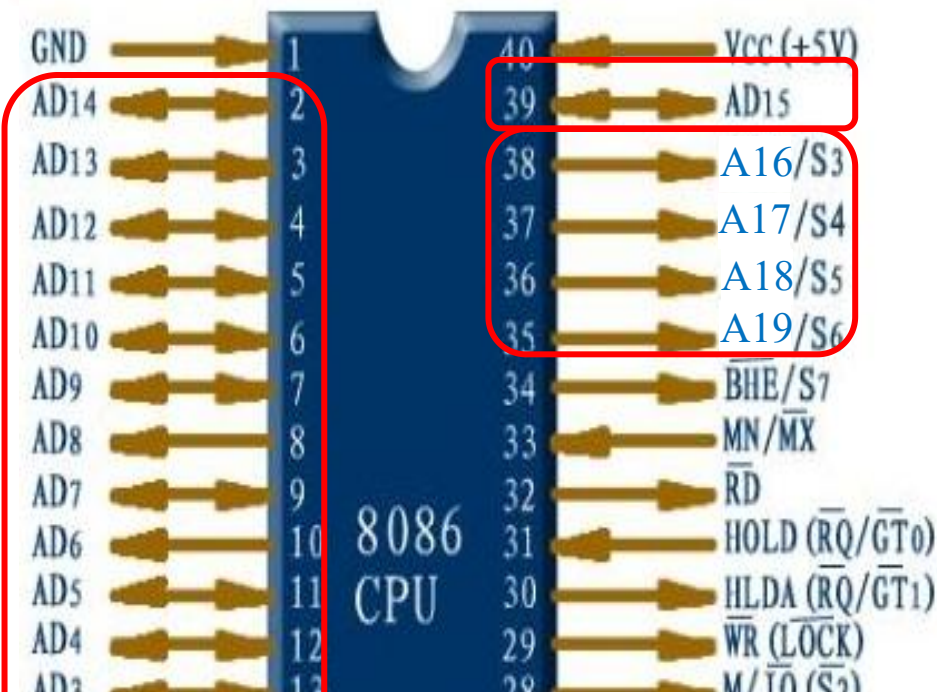
包含**两个或多个微处理器**，一个主处理器是8086，协处理器如数学运算协处理器8087、输入/输出协处理器8089

2. 1. 3 8086CPU的工作模式和引脚信号



Proteus 中8086CPU

2.1.3 8086CPU的工作模式和引脚信号



0 0: ES

0 1: SS

1 0: CS 或不需要使用段寄存器 (**I/O** 操作, 中断响应)

1 1: DS

● 8086CPU地址/数据线

◆ $AD_{15} \sim AD_0$: 地址/数据信号线 **分时复用**。传送地址信号时为输出，传送数据信号时为双向。

◆ $A_{19}/S_6 \sim A_{16}/S_3$: 地址/状态线信号 **分时复用**。

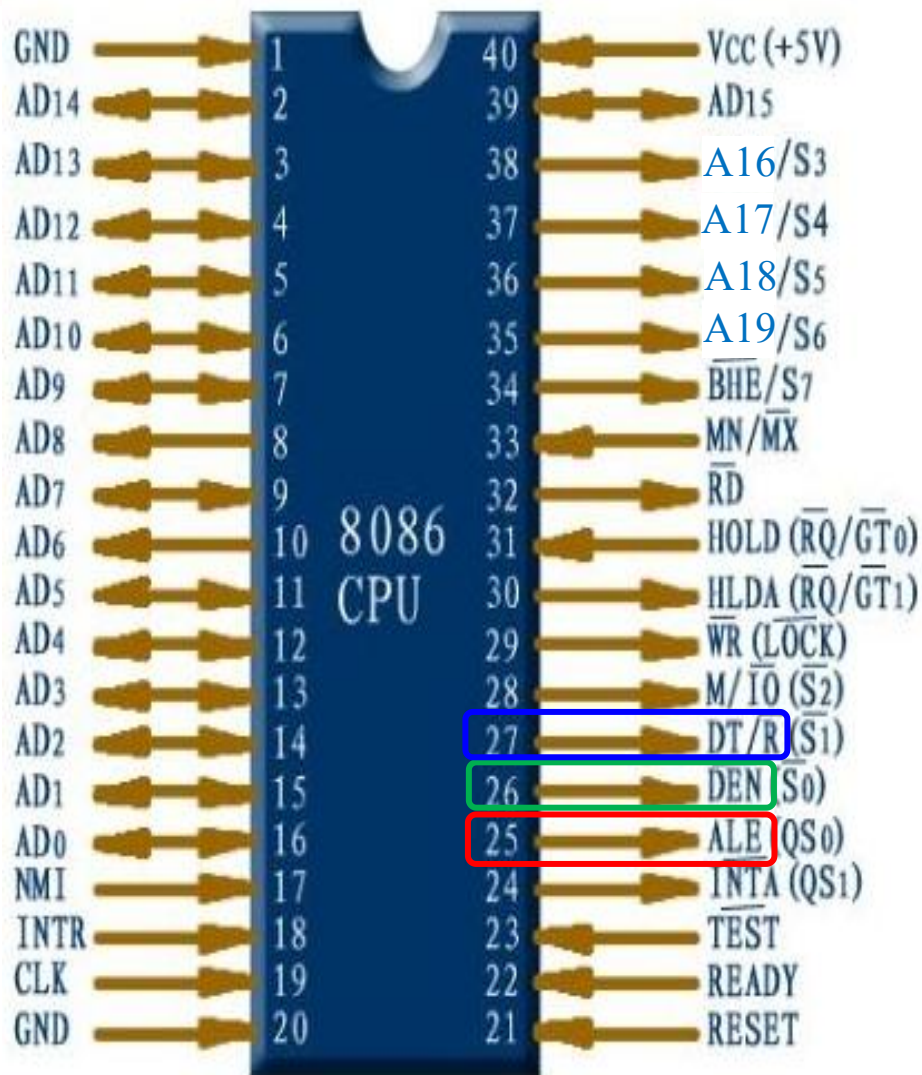
	$AD_{15} \sim AD_0$	$A_{19}/S_6 \sim A_{16}/S_3$
T1	输出地址信号	输出地址信号
T2~T4	传送数据信号	输出状态信号

S_6 : 恒为0，表示8086与总线相连

S_5 : 指示IF的当前状态

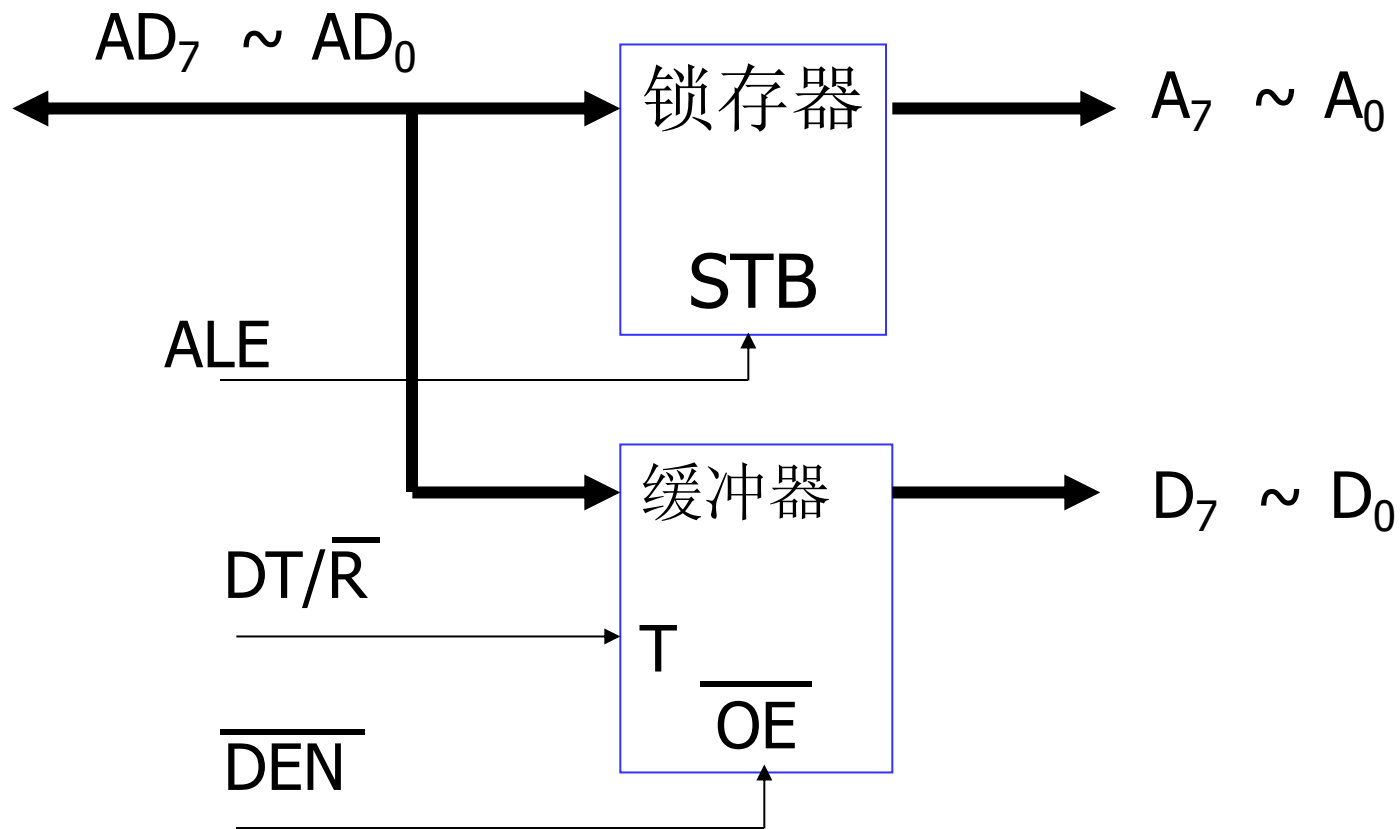
S_4S_3 : 合起来指示当前使用的段寄存器的情况

2.1.3 8086CPU的工作模式和引脚信号

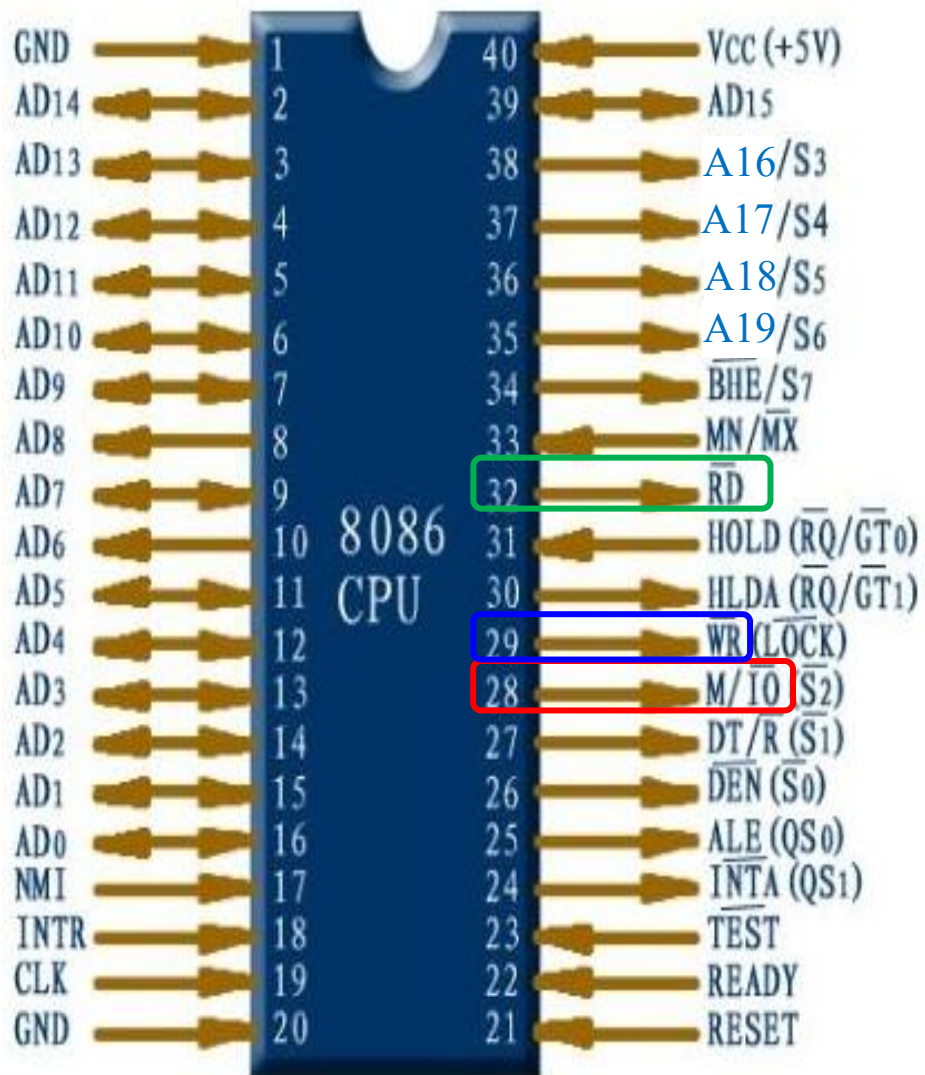


- **ALE**: 地址锁存允许信号
 - ◆ 高电平有效, **T1**状态有效, 将地址信息锁存到地址锁存器
- **DEN***: 数据允许信号
 - ◆ 三态, 输出, 低电平有效。 **T₂ ~ T₄** 状态有效, 用作数据收发器选通信号
- **DT/R***: 数据发送/接收控制信号, 三态, 输出
 - ◆ 控制数据传送的方向
 - DT/R* = 1**, CPU发送数据;
 - DT/R* = 0**, CPU接收数据

2.1.3 8086CPU的工作模式和引脚信号



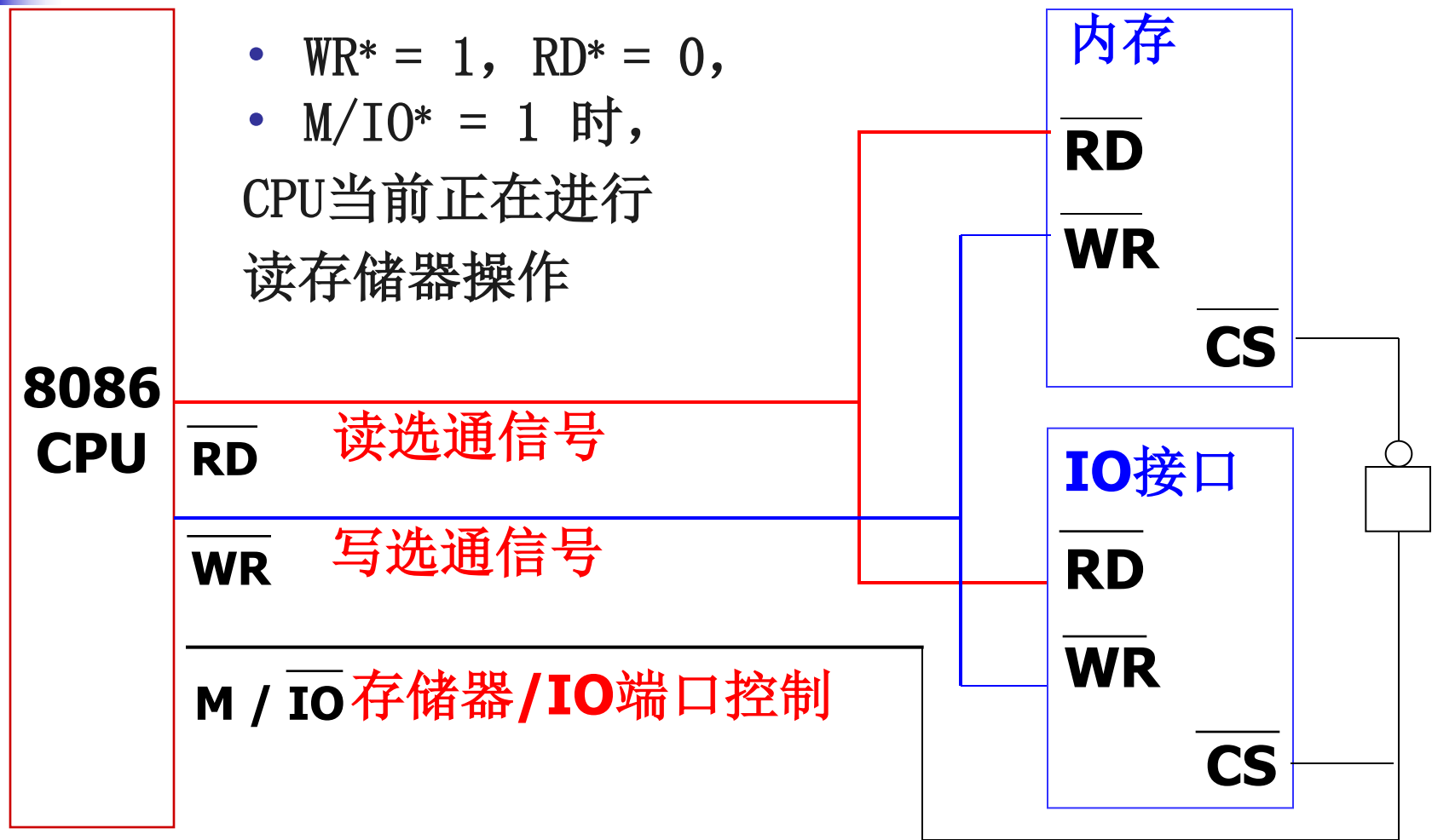
2.1.3 8086CPU的工作模式和引脚信号



- **M/IO***: 存储器/**IO**端口选择控制, 三态, 输出
- ◆ **M/IO*=1**, 访问存储器;
M/IO*=0, 访问**IO**端口;
- **RD***: 读信号
- ◆ 三态, 输出, 低电平有效。
- **WR***: 写信号
- ◆ 三态, 输出, 低电平有效。

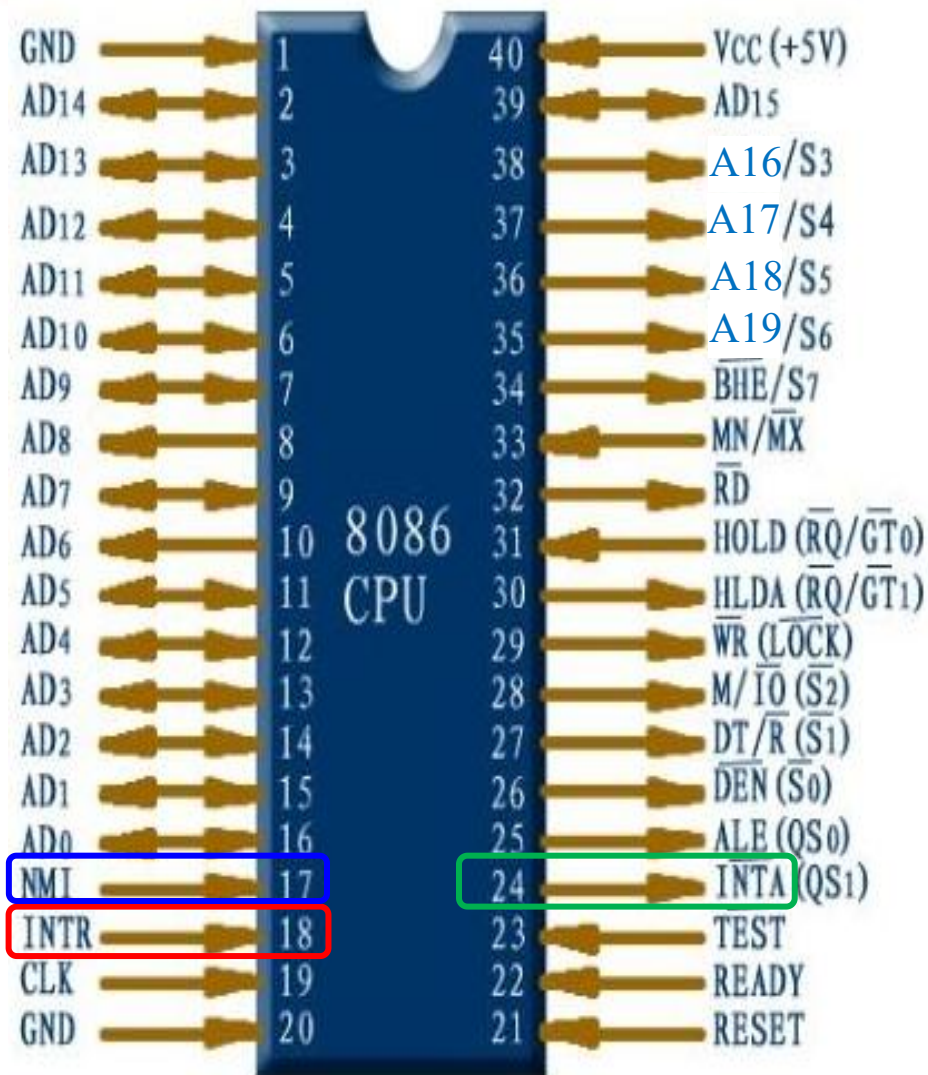
[例]: MOV AL, [2000H]

；功能：将存储器指定单元的数据传送给**AL**



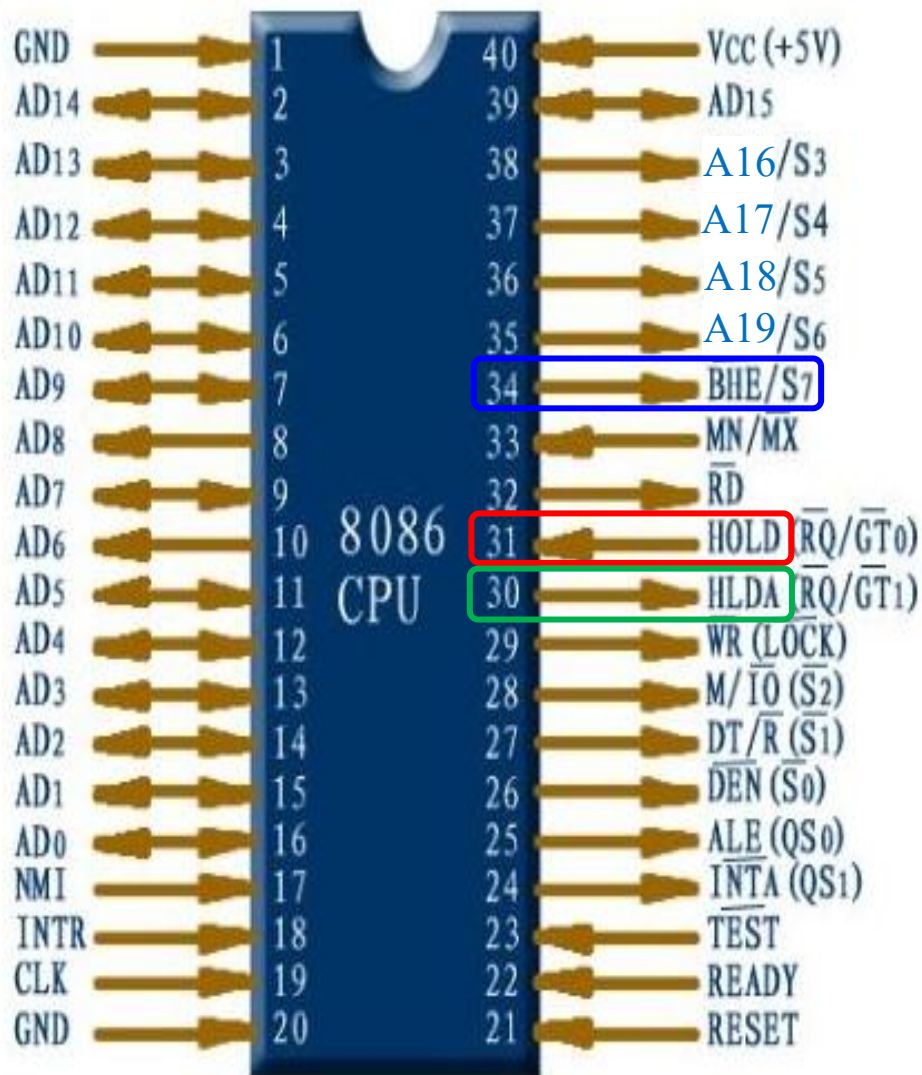
$\overline{\text{CS}}$ 为片选信号，低电平有效

2.1.3 8086CPU的工作模式和引脚信号



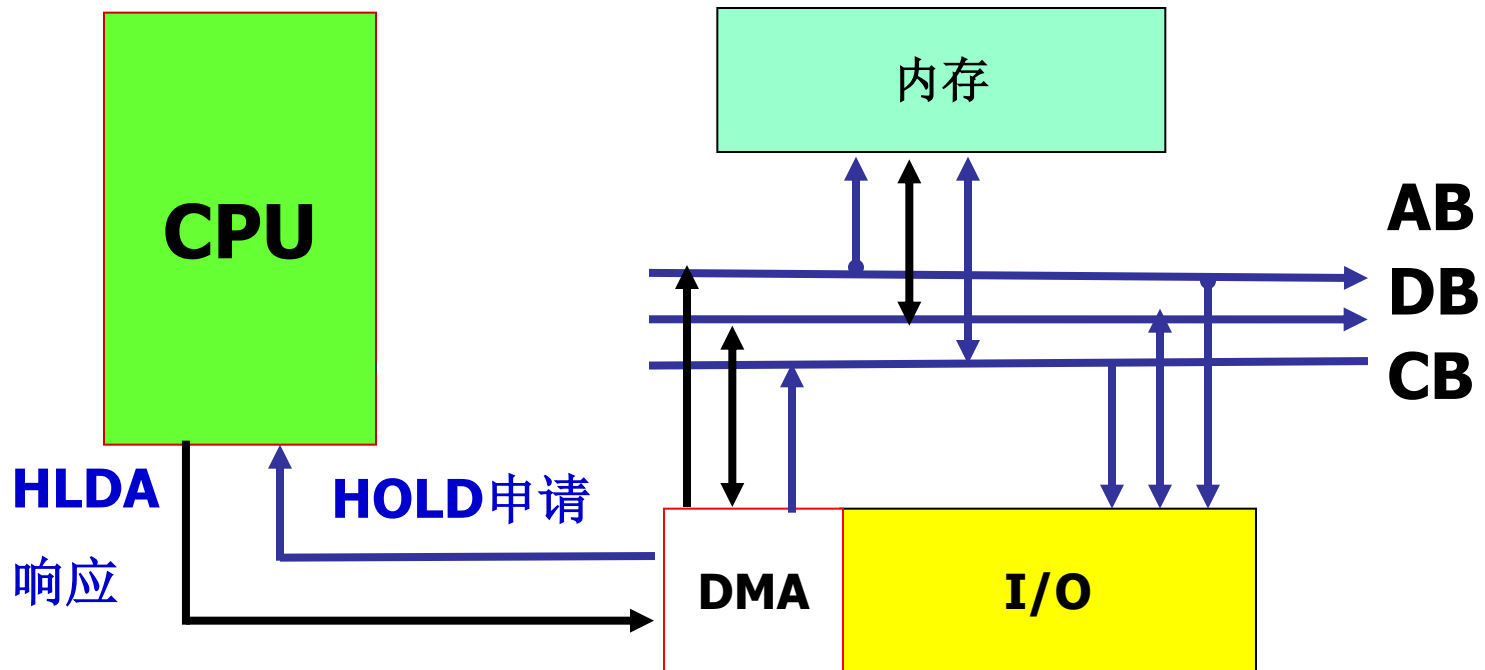
- **INTR:** 可屏蔽中断请求
 - ◆ 输入，高电平有效。有效时表示I/O设备向CPU申请中断
- **INTA*:** 中断响应信号
 - ◆ 输出，低电平有效。
 - IF=1**，允许响应
 - IF=0**，禁止中断
- **NMI:** 不可屏蔽中断请求
 - ◆ 输入，正跳变上升沿有效。

2.1.3 8086CPU的工作模式和引脚信号



- **HOLD**: 总线保持请求信号
◆ 输入，高电平有效。有效时，请求CPU让出总线控制权
- **HLDA**: 总线保持响应信号
◆ 输出，高电平有效。有效时，CPU让出总线控制权
- **BHE*/S7**: 高8位数据总线允许/状态信号
◆ 输出，三态，低电平有效。在T1状态有效，表示在高8位数据总线上传送1字节数据

例：DMA数据传送



思考：

- 每个存储单元用唯一的**20**位地址表示

物理地址

物理地址

										0 0000 H
...										0 0001 H
...										...
...										...
...										...
...										...
...										...
...										...
...										F FFFE H
...										F FFFF H

内存存储器
(1MB)

1个字节数据： (20000H) = 67H

1个字数据： (3000H) = 1122H

- 占用连续的存储单元;
- 低字节数据存放在低地址, 高字节数据存放在高地址中, 如内存**20000H**地址中存放双字数据**01234567H**

- 低字节数据从偶地址开始存放
- 如数据**1122H**存放在**30000H**开始的单元中

- 低字节数据从**奇**地址开始存放
- 如数据**3344H**存放在**40001H**开始的单元中

									00000H
...									...
67H									20000H
45H									20001H
23H									20002H
01H									20003H
...									
22H									30000H
11H									30001H
...									
44H									40001H
33H									40002H
...									

(2) 内存存储器的分段

问题：8086CPU内部寄存器是16位，最多寻址64KB的空间，如何实现对1MB空间的寻址？（即20位的物理地址如何用16位的寄存器表示呢？）

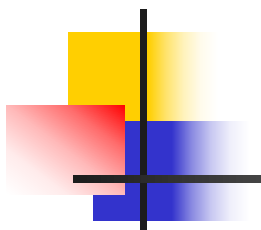
方法：存储空间分段技术

- 具体实现：将**1MB**分成若干个逻辑段，每段最大为**64KB**，且段起始地址可以被**16**整除（即后四位为**0**）。

逻辑段

- 代码段、数据段、附加段和堆栈段

内存存储器 (1MB)								物理地址
								0 0000 H
...								0 0001 H
...								...
...								...
...								...
...								...
...								...
...								...
...								F FFFE H
...								F FFFF H



2.1.4 8086的内存存储器和I/O端口组织

逻辑地址和物理地址

逻辑地址——段地址：偏移地址
物理地址 = 段地址 × 16 + 偏移地址

6A00H	5D00H
+ 2200H	+ F200H
6C200H	6C200H

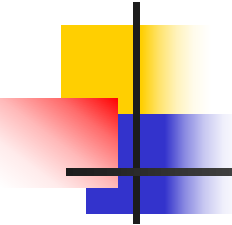
思考：

- 存储单元的物理地址是唯一的吗？
- 存储单元的逻辑地址是唯一的吗？

例如：

- 已知CS=6A00H，偏移地址=2200H，求物理地址
- 已知CS=5D00H，偏移地址=F200H，求物理地址

存储单元的物理地址是唯一，而逻辑地址不唯一。



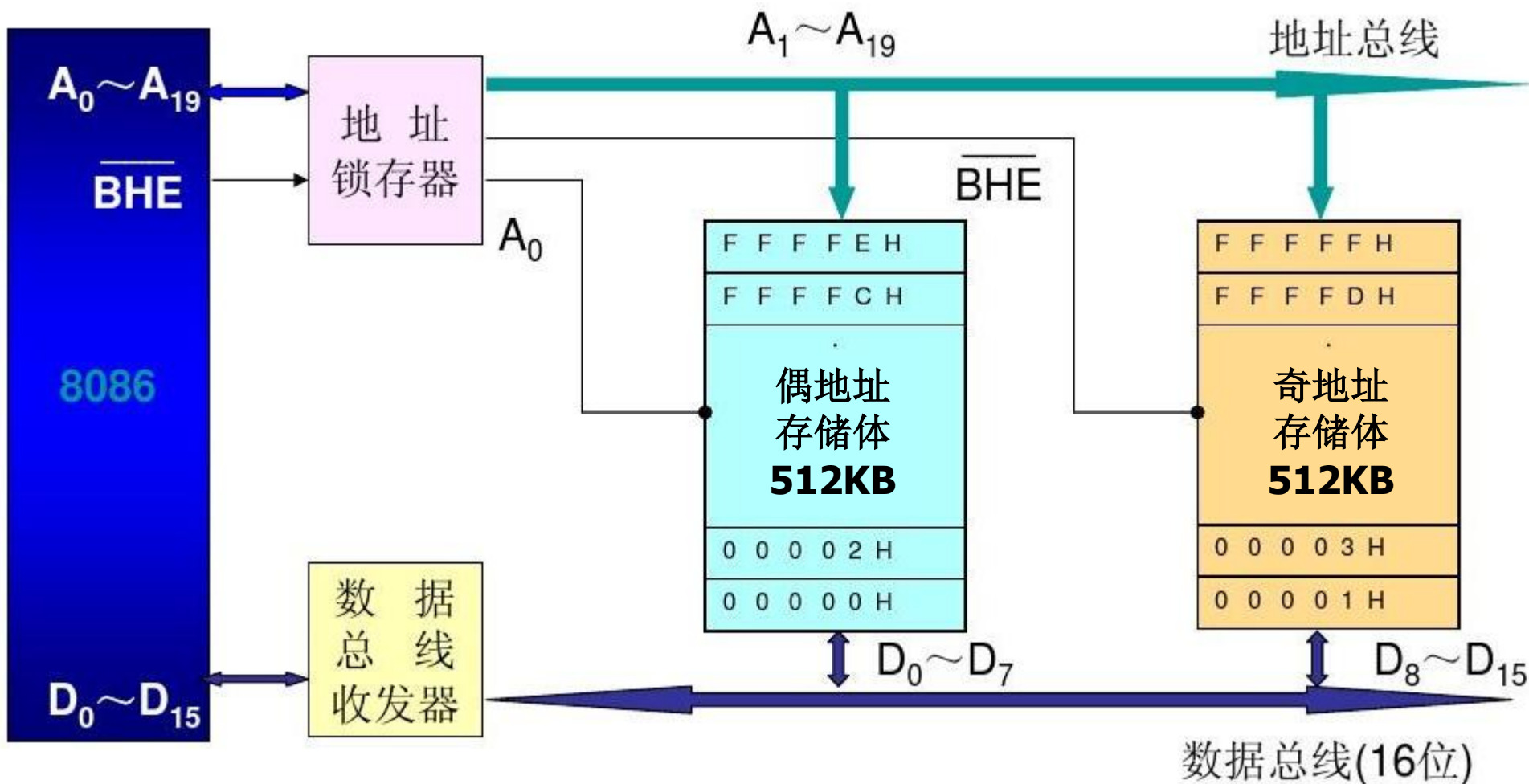
2.1.4 8086的内存储器和I/O端口组织

对访问不同逻辑段，逻辑地址使用寄存器的约定

逻辑段	段地址	偏移地址	
代码段	CS	IP	
数据段	DS	BX/SI/DI	偏移地址由指令的寻址方式得到的有效地址（ EA ）确定
附加段	ES	BX/SI/DI	
堆栈段	SS	SP/BP	

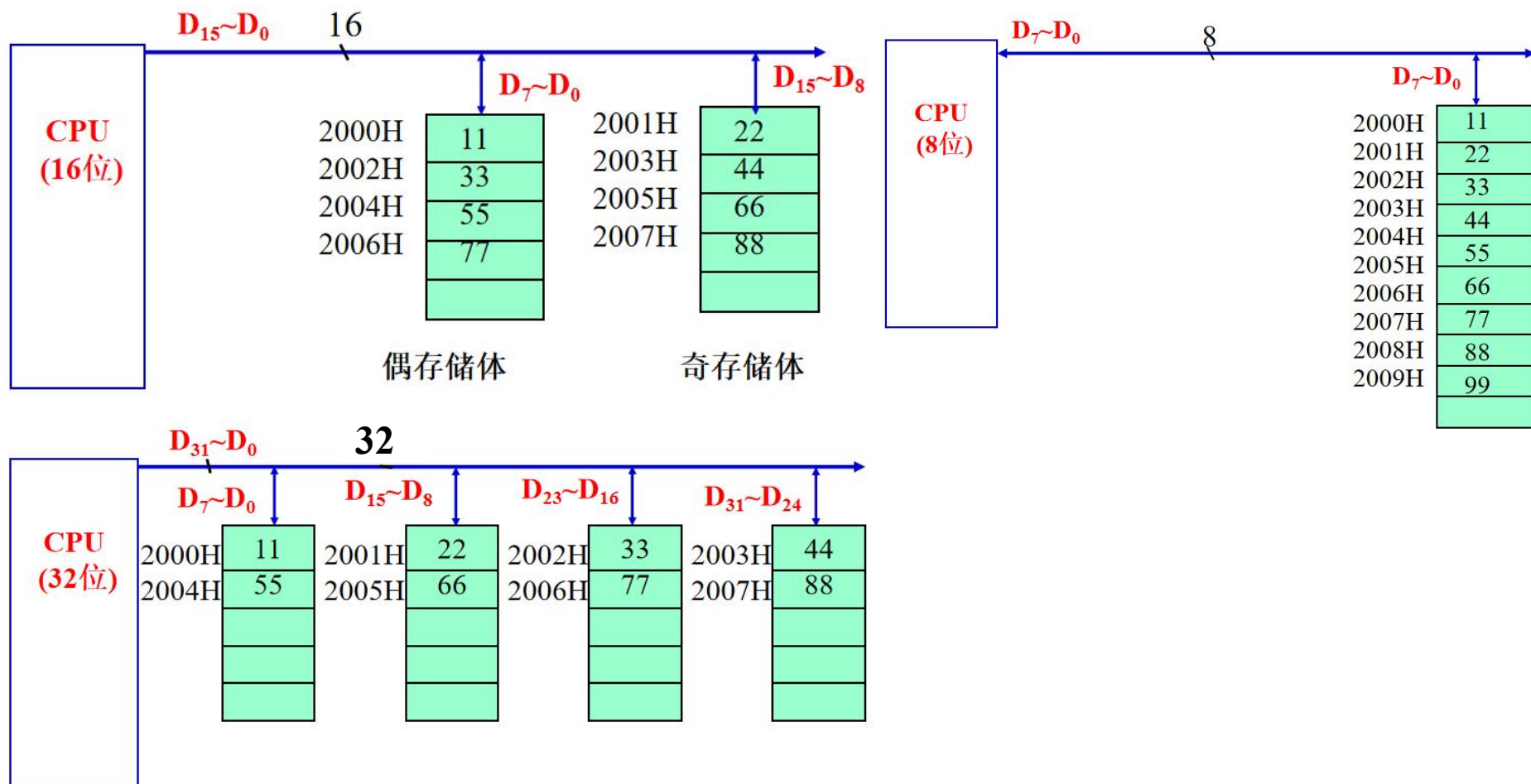
2.1.4 8086的内存储器和I/O端口组织

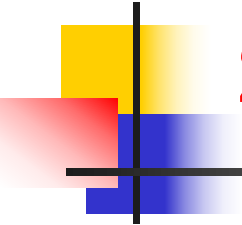
2. 8086的内存储器结构——分体结构字节交叉编址（自学）



2.1.4 8086的内存储器和I/O端口组织

2. 8086的内存储器结构——分体结构字节交叉编址（自学）

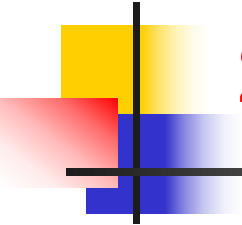




2.1.4 8086的内存储器和I/O端口组织

3. 8086的I/O端口

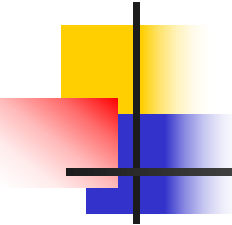
- 在8086微机系统中，输入 / 输出设备必须通过输入 / 输出接口芯片与CPU相连。
- 每个I/O接口芯片都有一个或几个I/O端口（端口指接口中可被CPU直接读 / 写的寄存器），每个I/O端口都有一个唯一的端口地址。
- 8086用地址总线的低16位 $A_{15} \sim A_0$ 来确定端口地址，共64 K 个，地址范围为0000H~FFFFH。
- 访问I/O端口时：M/IO*=0



2.1.4 8086的内存存储器和I/O端口组织

4. 8086专用存储区

- 00000H—003FFH共 1 KB单元，存放中断向量，该区域称为**中断向量表**；
- FFFF0H-FFFFFFH共16个单元，存放一条**无条件转移指令**，将系统转入系统初始化程序，当系统加电或复位后可自动进入系统程序。



2.1.5 8086最大、最小模式系统的构成

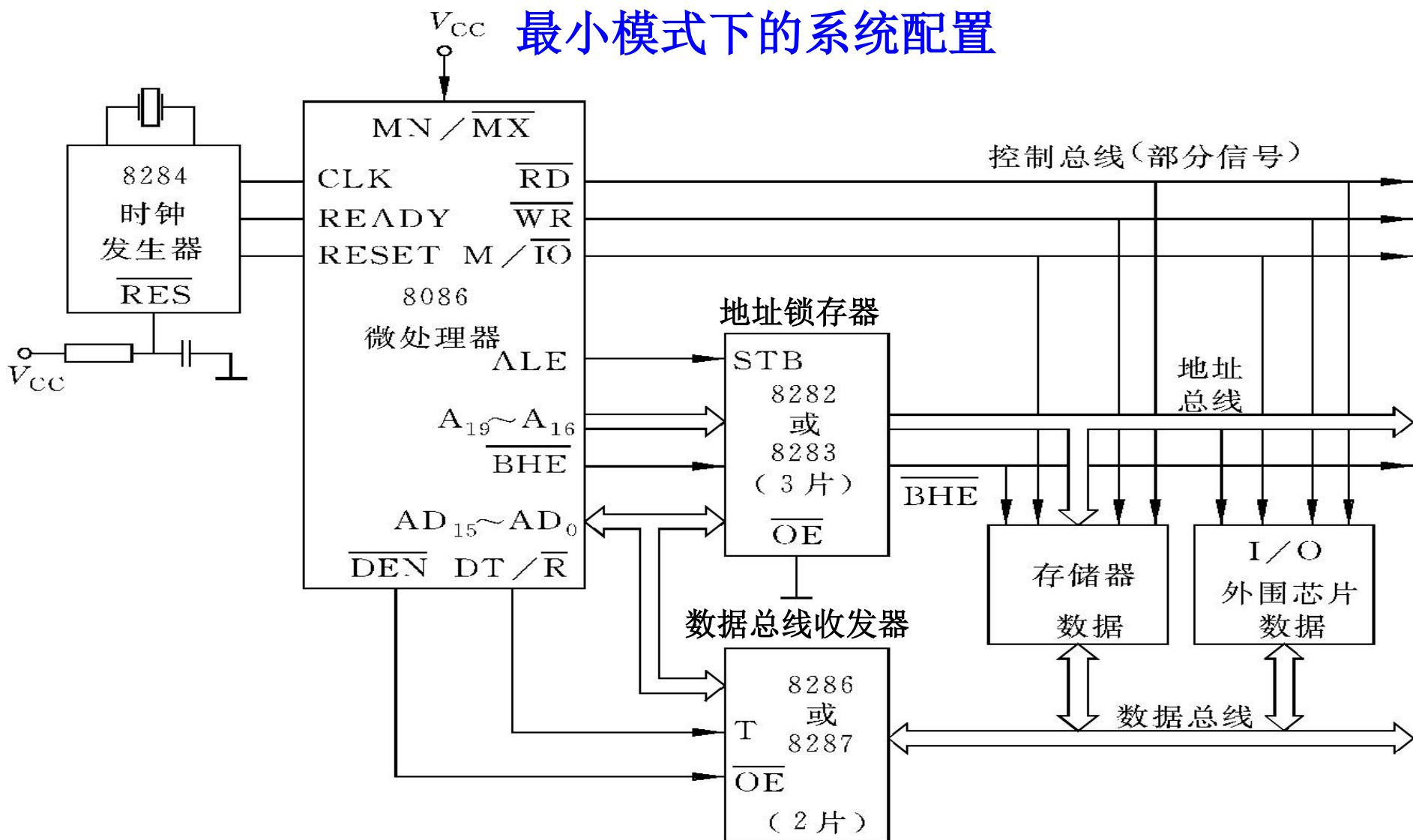
1. 最小模式和最大模式的区别

最小模式	最大模式
MN/MX* 接+5V 构成单处理器系统 系统控制信号由 CPU 产生	MN/MX* 接地 构成多处理器系统 系统控制信号由总线控制器产生

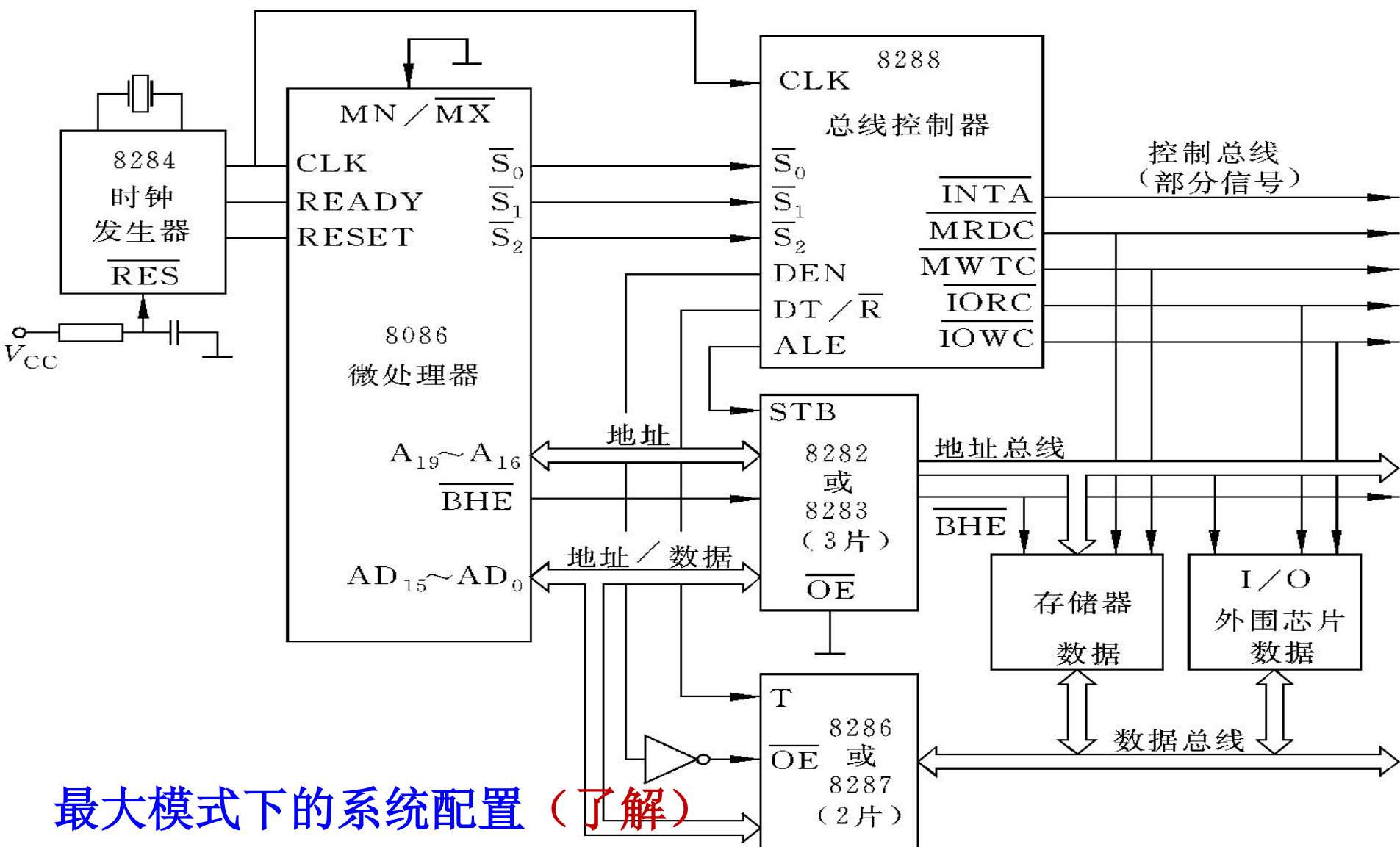
多处理器系统：主处理器为**8086**，其他的处理器称为协处理器，如数学协处理器**8087**，输入/输出协处理器**8089**。

2.1.5 8086最大、最小模式系统的构成

最小模式下的系统配置



2.1.5 8086最大、最小模式系统的构成

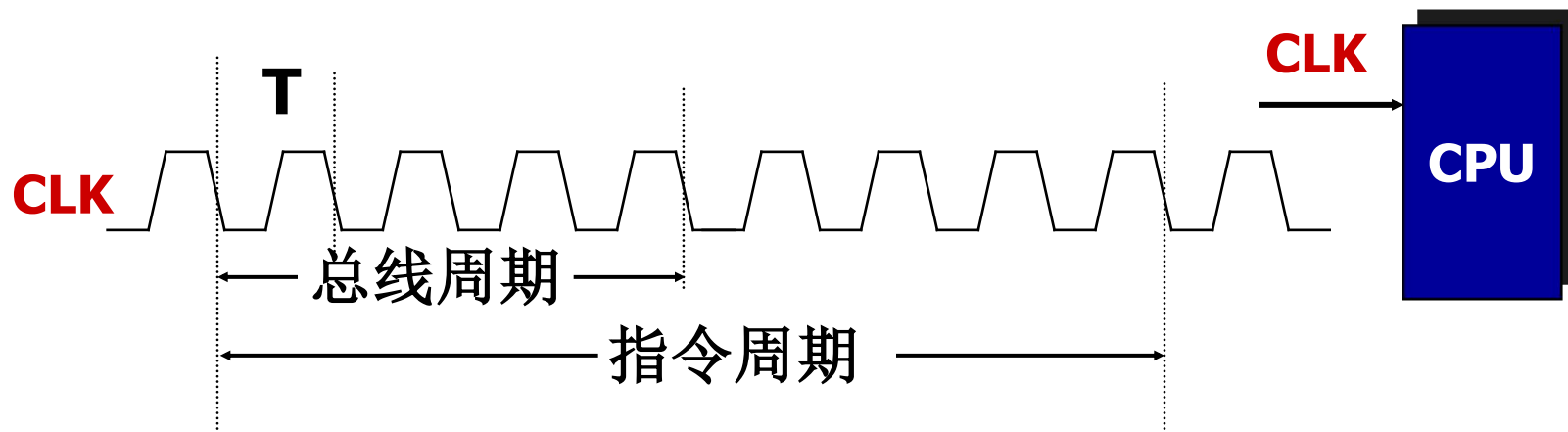


最大模式下的系统配置 (了解)

2.1.6 8086CPU的工作时序

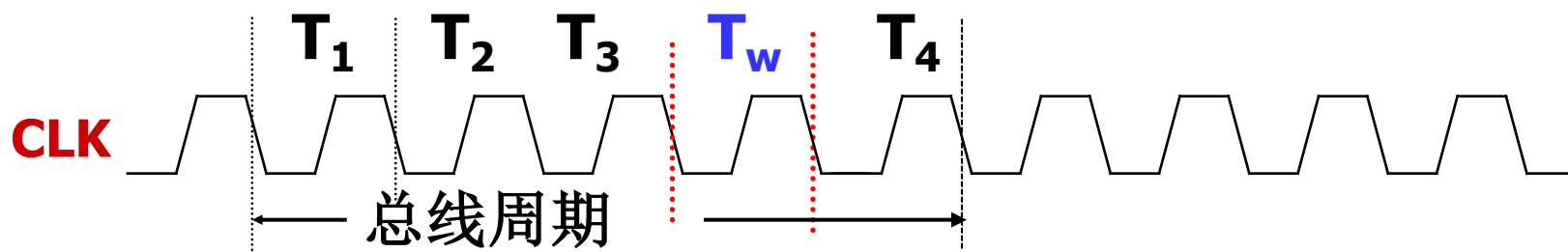
1. 时钟周期、总线周期、指令周期

- 时序：**CPU**各引脚信号在时间上的关系



- 时钟周期**：每个时钟脉冲持续时间，又称**T状态**。
- 总线周期**：对存储器或**I/O**端口进行一次读/写所需的时间；
- 指令周期**：执行一条指令所需的时间，分为**若干**总线周期；
- 8086CPU时钟频率为5MHz，一个T状态为200ns
- 8086一个总线周期至少包括4个时钟周期 $T_1 \sim T_4$

2.1.6 8086CPU的工作时序



- **等待周期:** 在基本总线周期内不能完成读/写, 则在 T_3 和 T_4 间插入等待周期 T_w 。

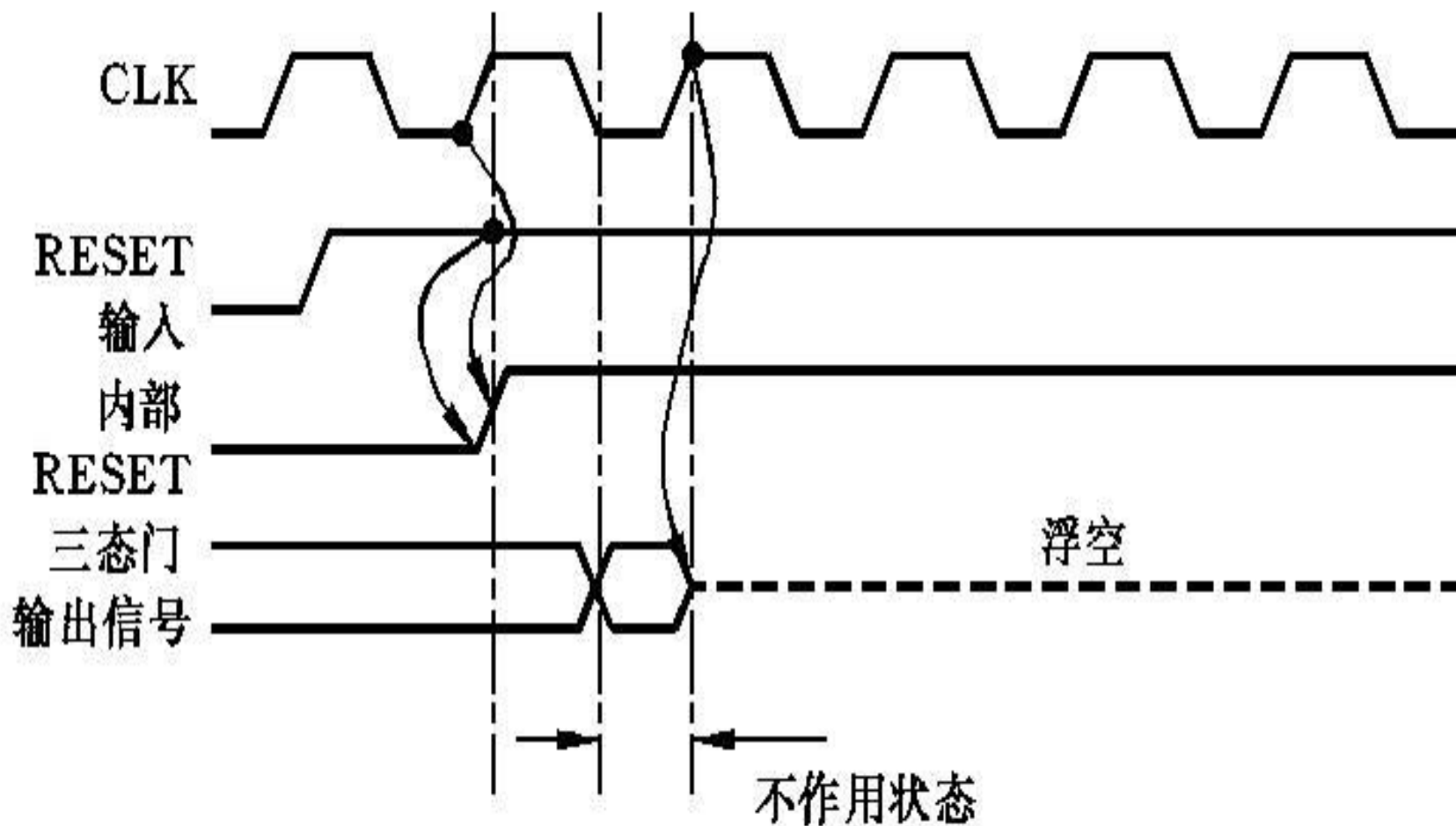
2 总线操作时序

8086的主要操作时序有以下几种:

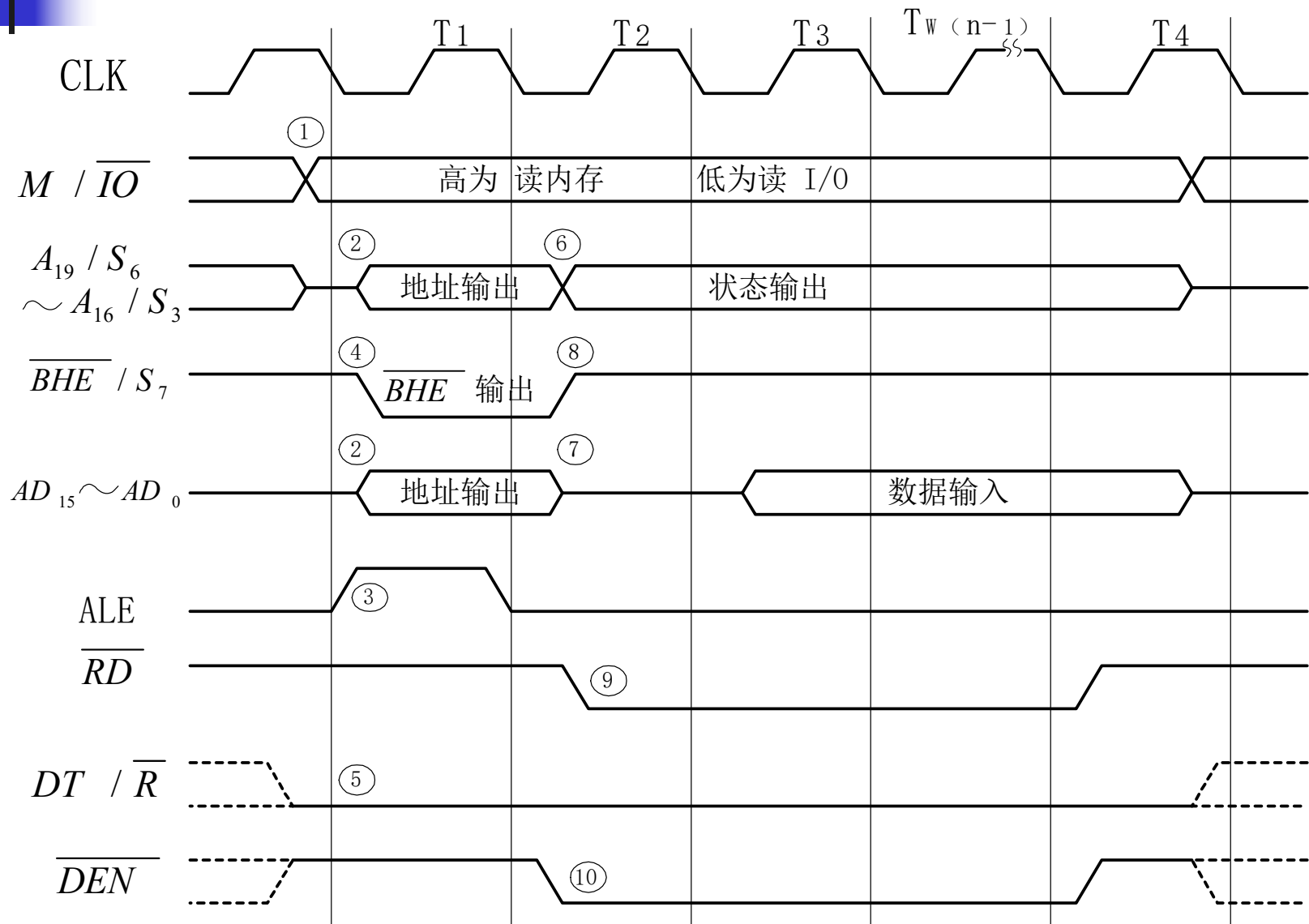
- ①系统的复位和启动
- ②最小模式下的读总线周期、写总线周期
- ③最小模式下的总线请求和响应周期
- ④最大模式下的读总线周期、写总线周期
- ⑤最大模式下的总线请求/允许周期

2. 1. 6 8086CPU的工作时序

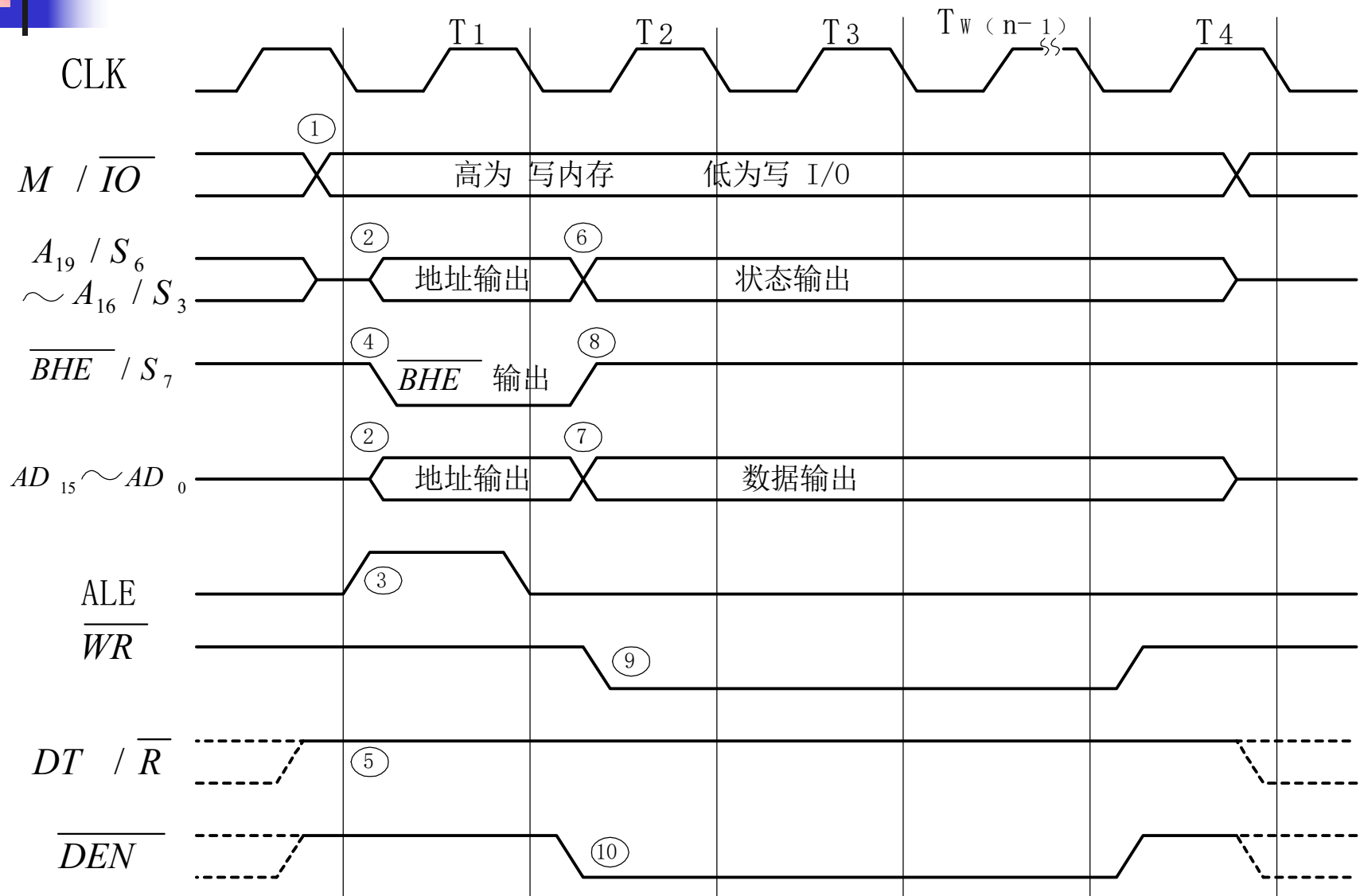
3. 系统的复位和启动——复位操作时序



4. 时序分析--最小模式下存储器读总线周期



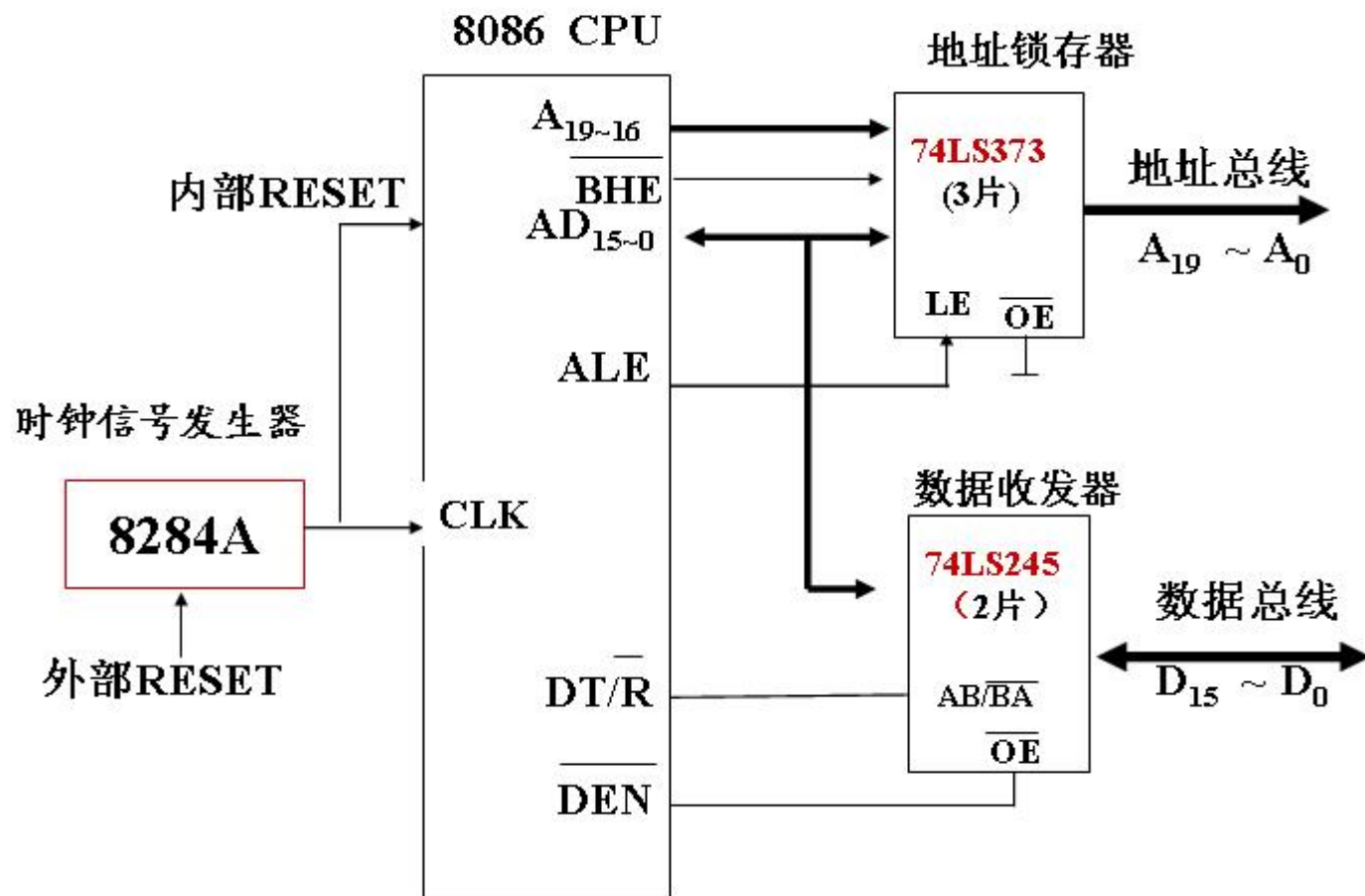
5. 时序分析--最小模式下存储器写总线周期

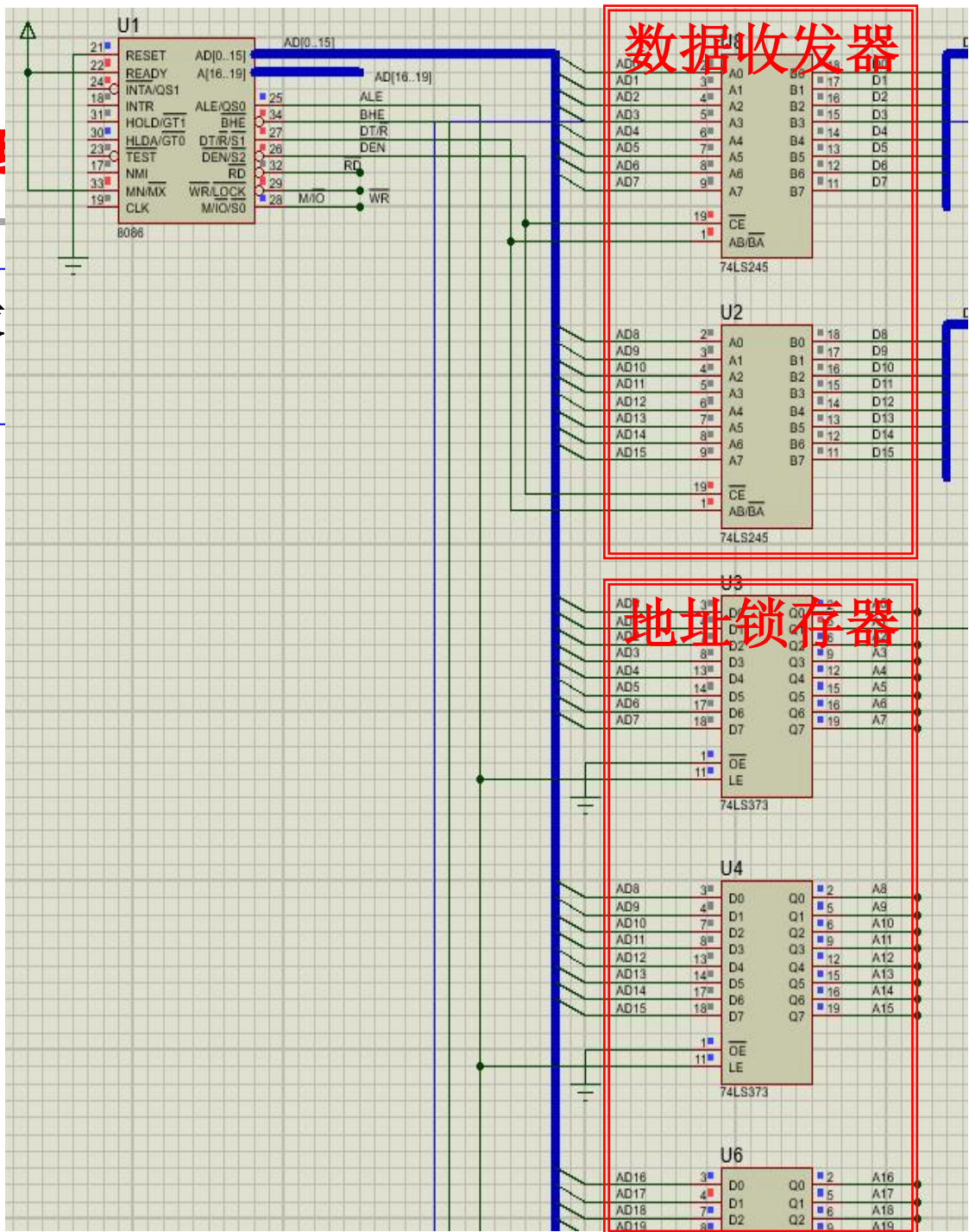


2.2 案例实现

案例2.1: 8086 CPU最小系统设计

◆ 8086CPU最小系统中系统总线的形成

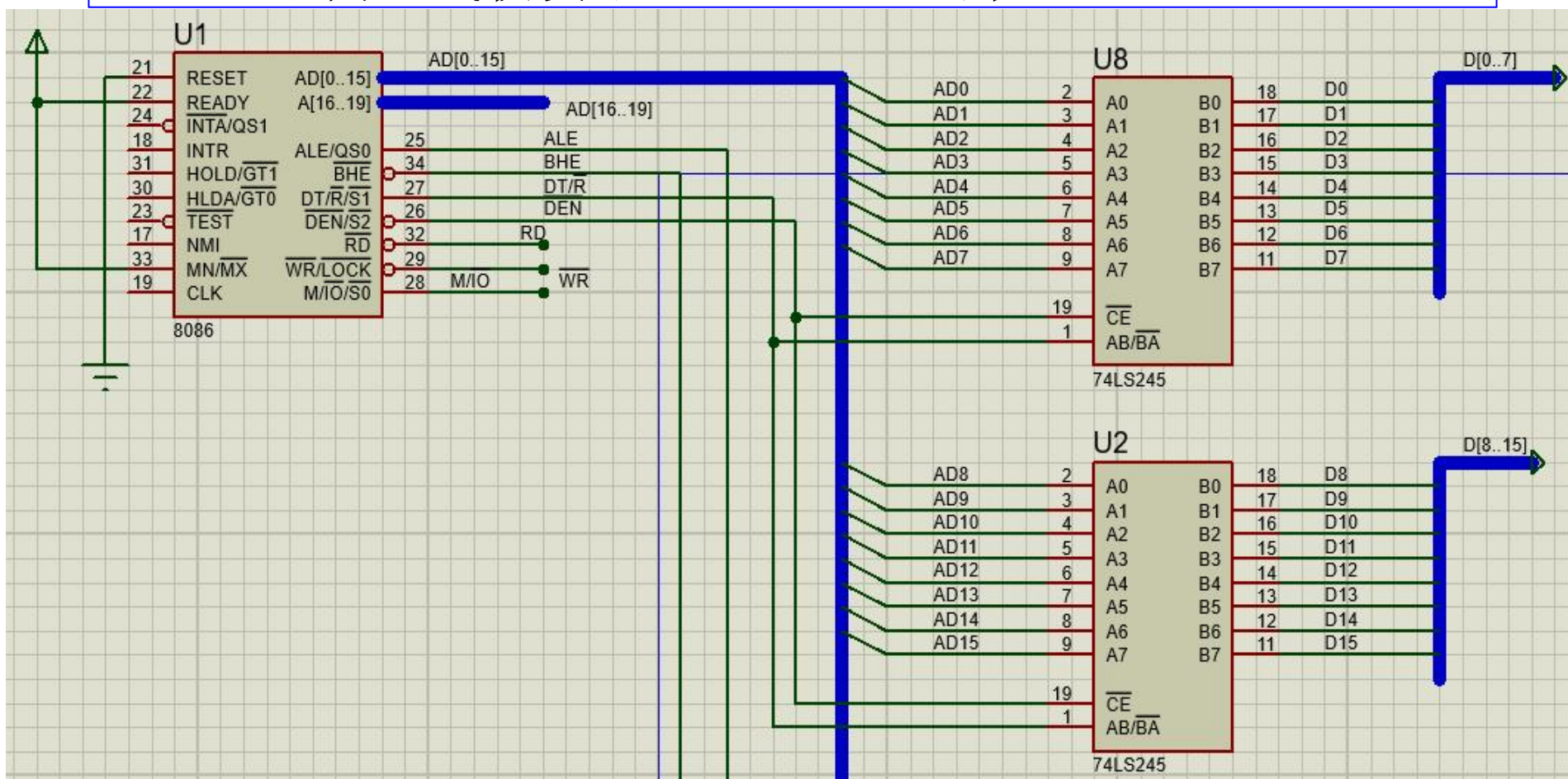




2.2 案例实现

案例2.1: 8086 CPU最小系统设计

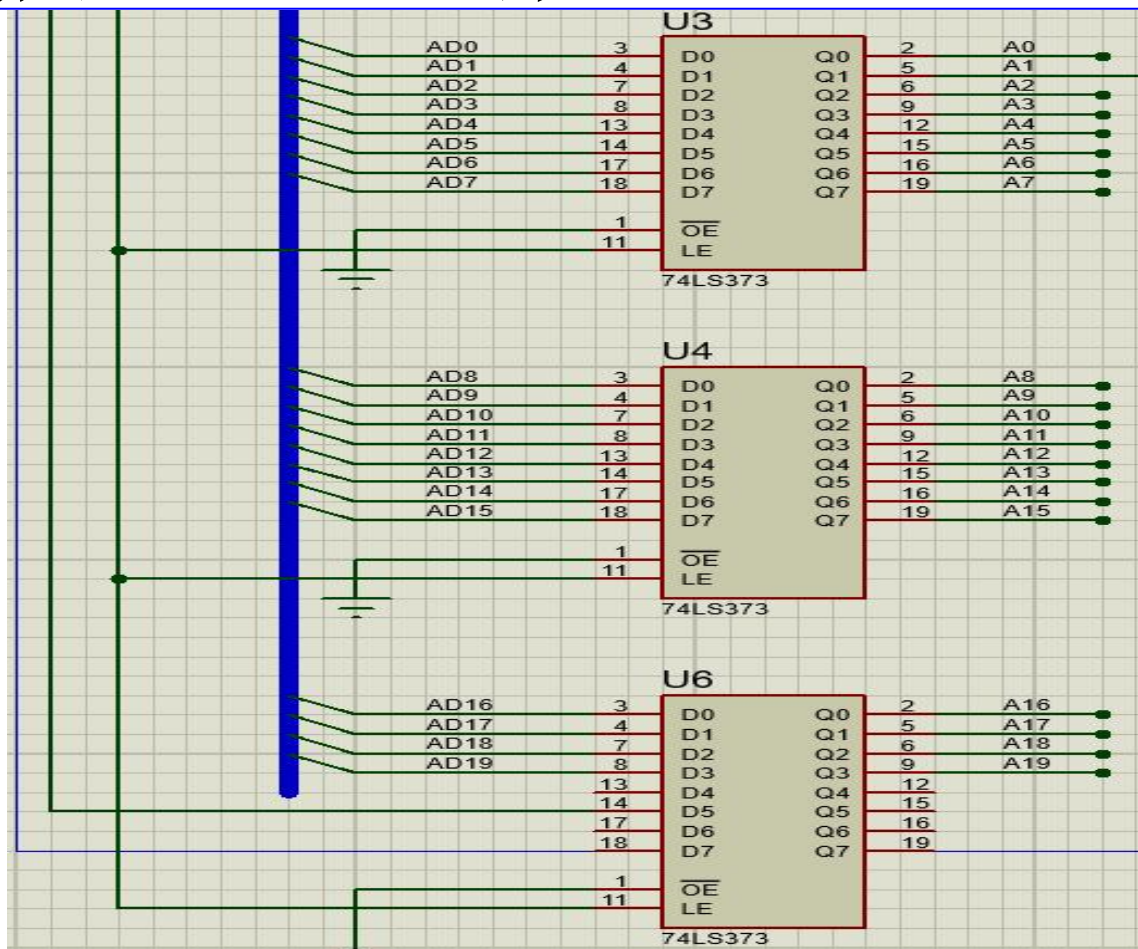
◆ 三态双向总线收发器74LS245 (2片)



2.2 案例实现

案例2.1: 8086 CPU最小系统设计

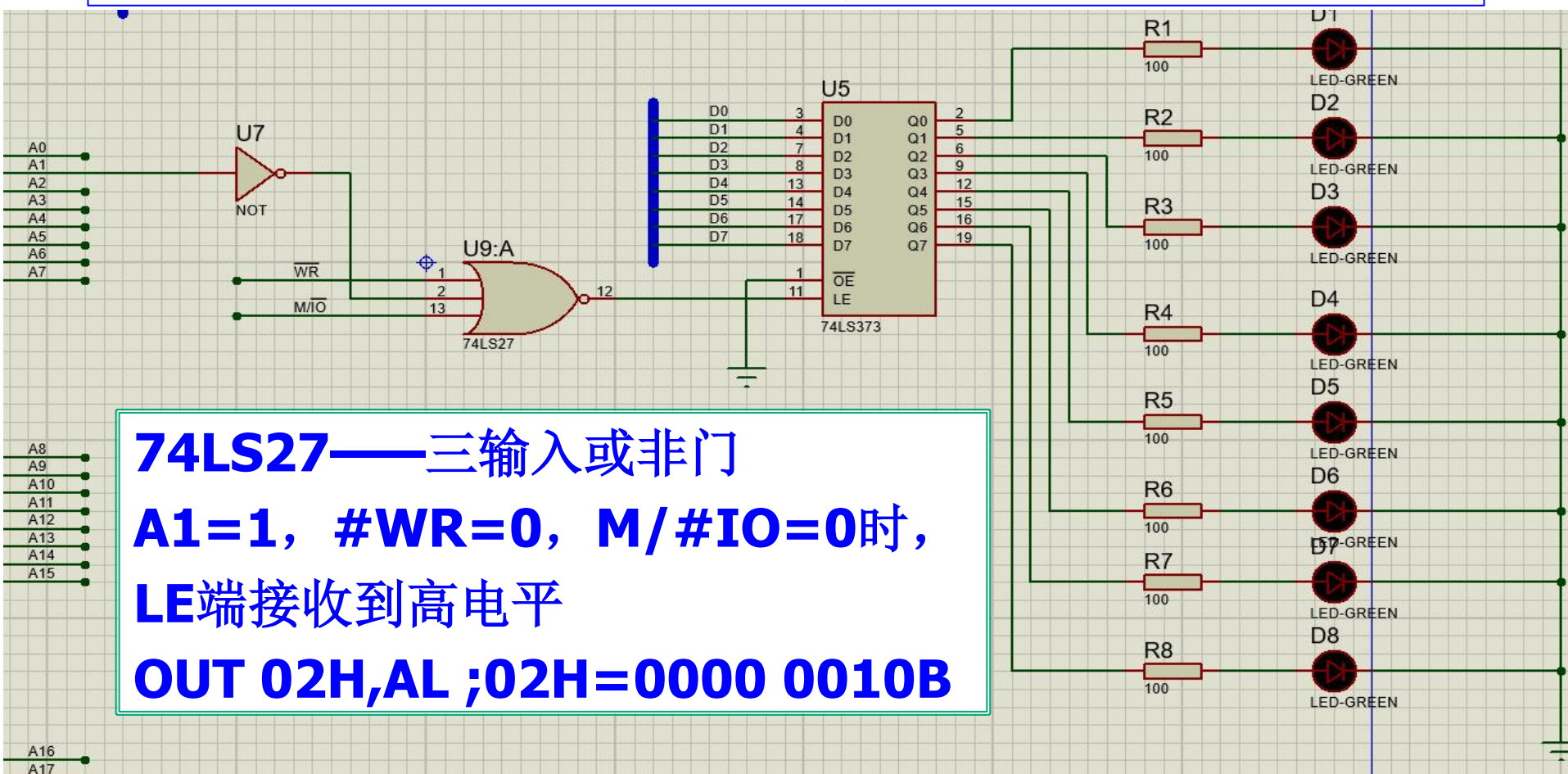
◆ 地址锁存器74LS373 (3片)



2.2 案例实现

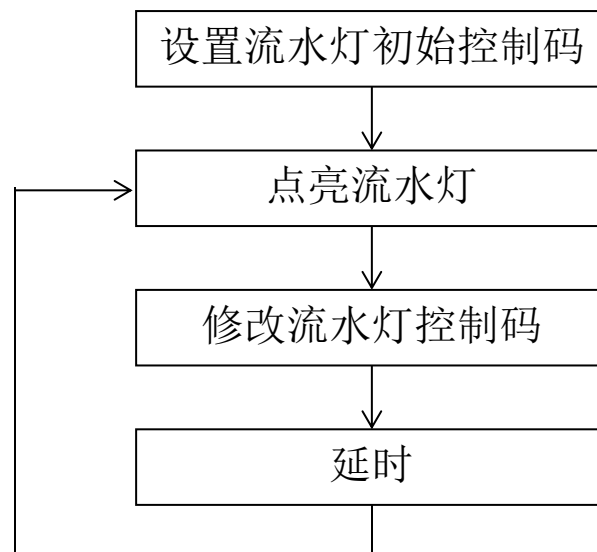
案例2.1: 8086 CPU最小系统设计

◆ 地址译码器74LS27和数据锁存器74LS373



案例2.1：设计8086 CPU最小工作模式下的系统配置，并实现简单流水灯。（流程图和参考程序）

```
12 CODE    SEGMENT PUBLIC 'CODE'
13         ASSUME CS:CODE
14 START:
15         ; Write your code here
16         MOV BL,00000001B ;设置流水灯初始控制码, 点亮第1个灯
17 ENDLESS:MOV AL,BL       ;若不为0, 则开关为打开状态, 输出流水灯控制码BL的值
18         OUT 02H,AL
19         ROL BL,01H       ;流水灯控制码左循环1位, 即下一个灯亮, 其余灯灭
20         CALL DELAY100    ;调用延时子程序, 软件延时约1秒
21         JMP ENDLESS
22 DELAY100 PROC           ;延时子程序, CPU执行下面程序耗时约1s
23         PUSH CX
24         MOV CX,0
25         LOOP $
26         LOOP $
27         LOOP $
28         MOV CX,15000
29         LOOP $
30         POP CX
31         RET
32 DELAY100 ENDP
33 CODE     ENDS
34         END START
```



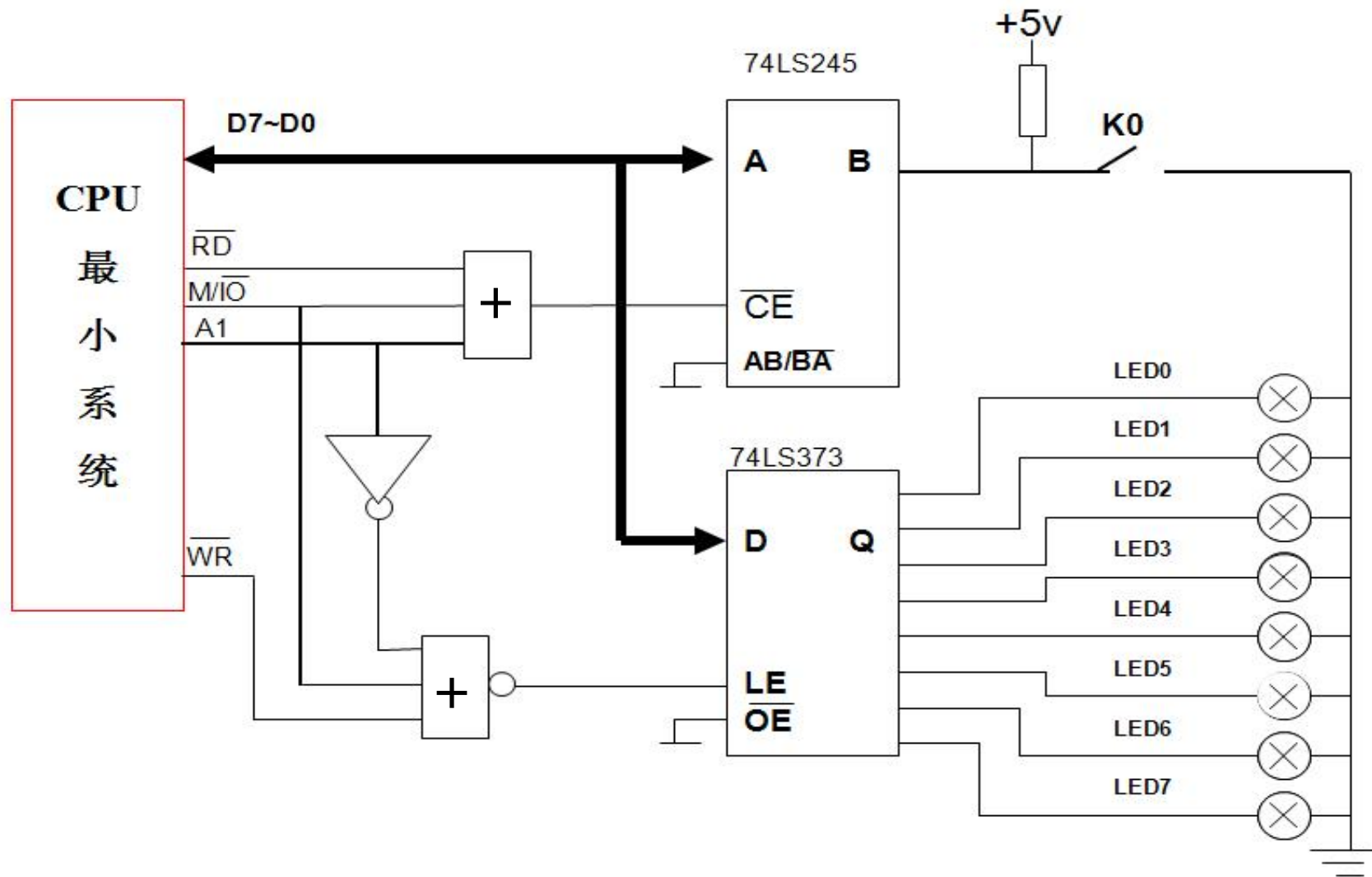
案例2.2：8086 CPU在流水灯控制系统中的应用

要求：在8086最小工作模式下，对流水灯进行控制。

当开关K0闭合（K0=0，接地），8086 CPU使流水灯全部熄灭；

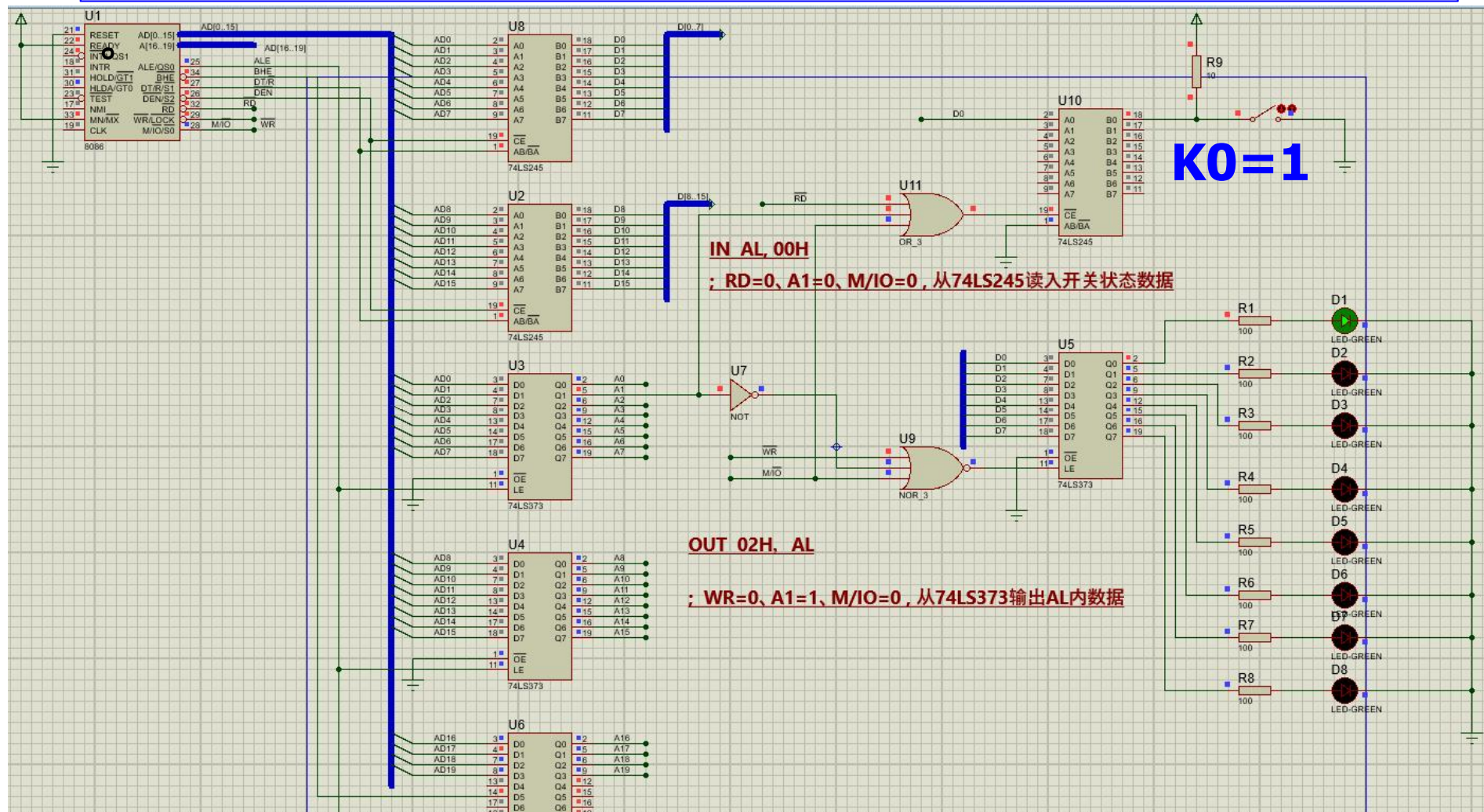
当开关K0断开时（K0=1，接入高电平），8086 CPU使流水灯依次点亮。

给出流水灯控制系统的原理图及控制程序，并仿真实现其功能。



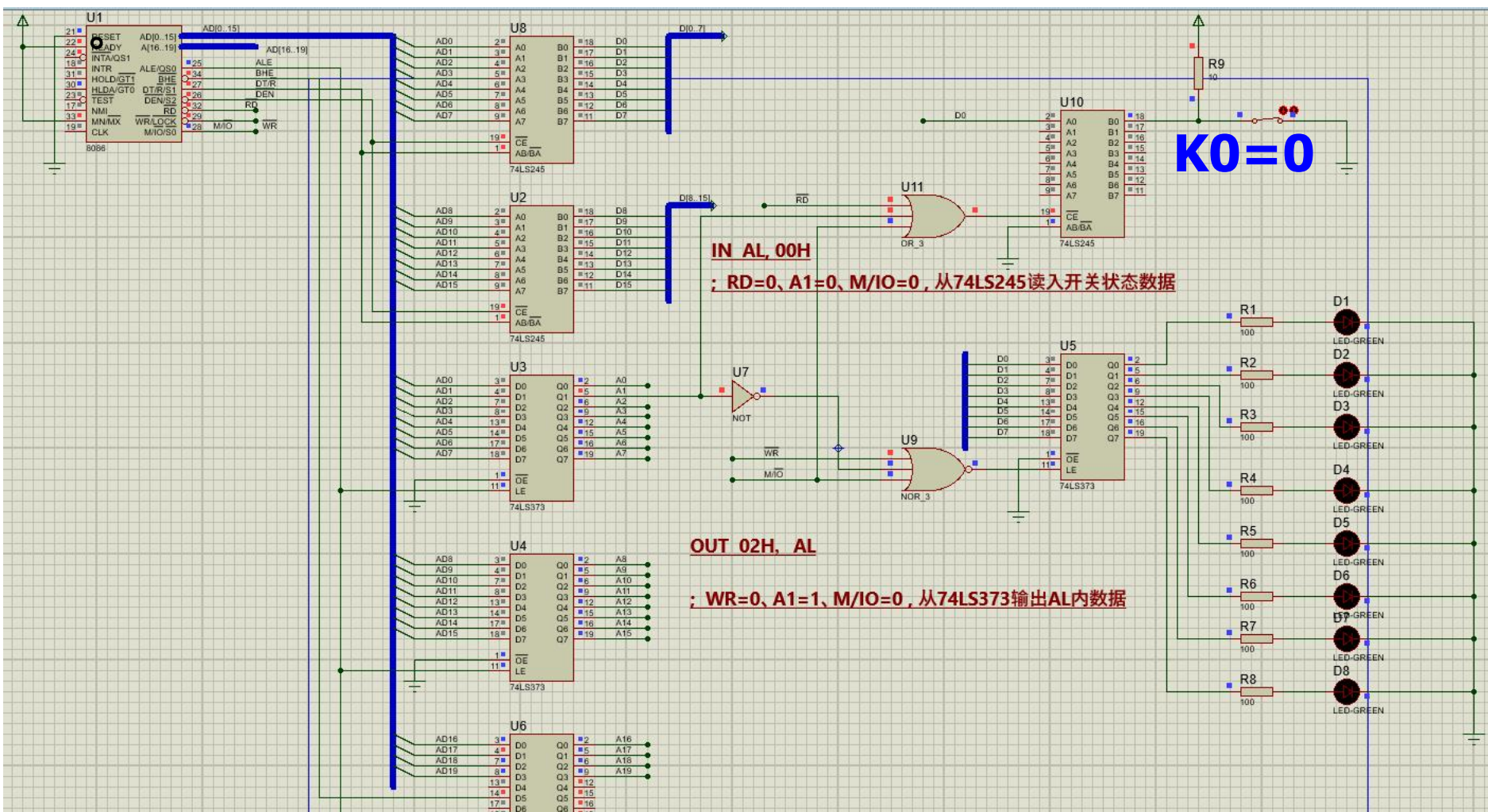
2.2 案例实现

案例2.2: 8086 CPU在流水灯控制系统中的应用——原理图

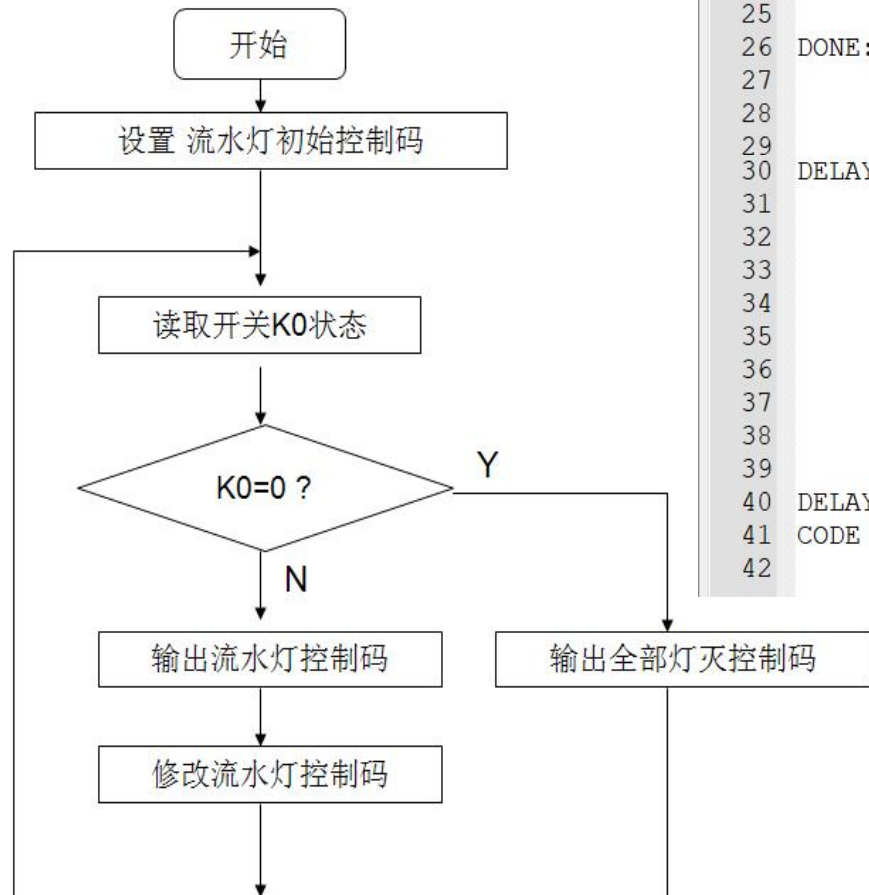


2.2 案例实现

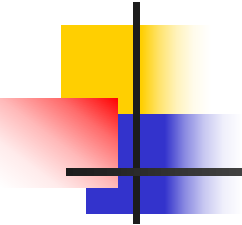
案例2.2: 8086 CPU在流水灯控制系统中的应用——原理图



案例2.2：参考程序和流程图



```
12 CODE SEGMENT PUBLIC 'CODE'
13     ASSUME CS:CODE
14 START:
15     ; Write your code here
16     MOV BL,00000001B ;设置流水灯初始控制码, 点亮第1个灯
17 ENDLESS: IN AL,00H ;读取开关状态, 00H为输入端口地址
18     AND AL,01H ;仅保留K0对应的数据位
19     CMP AL,00H ;判断开关K0是否为0, K0=0闭合
20     JZ DONE ;如果开关为0, 转 DONE 流水灯全灭;
21     MOV AL,BL ;若不为0, 则开关为打开状态, 输出流水灯控制码BL的值
22     OUT 02H,AL
23     ROL BL,01H ;流水灯控制码左循环1位, 即下一个灯亮, 其余灯灭
24     CALL DELAY100 ;调用延时子程序, 软件延时约1秒
25     JMP ENDLESS
26 DONE: MOV AL,00H
27     OUT 02H,AL ;使LED灯全灭
28     CALL DELAY100
29     JMP ENDLESS ;下一次过程
30 DELAY100 PROC ;延时子程序, CPU执行下面程序耗时约1s
31     PUSH CX
32     MOV CX,0
33     LOOP $
34     LOOP $
35     LOOP $
36     MOV CX,15000
37     LOOP $
38     POP CX
39     RET
40 DELAY100 ENDP
41 CODE ENDS
42 END START
```



Proteus软件操作说明

1. 8086芯片的使用及调试步骤

- (1) 新建工程
- (2) 界面介绍
- (3) 原理图绘制
- (4) 源代码
- (5) 编译、调试

(1) 新建工程

UNTITLED - Proteus 8 Professional - 主页

文件(F) 系统(Y) 帮助(H)

主窗体显示 Proteus DESIGN SUITE 8.4 的启动界面。

使用教程

- 原理图绘制
- PCB板设计
- 仿真
- 旧版本移植向导
- What's New

帮助中心

- 帮助主页
- 原理图绘制
- PCB板设计
- 仿真

关于软件

(C) Labcenter Electronics 1989-2015
中文版由广州市风标电子技术有限公司提供
发布版本 8.4 SP0 (内部版本21079) 具有高级仿真功能
www.labcenter.com

注册给:
FreeWare
Labcenter Electronics Ltd
用户编号: 17-56753-440
Network Licence Subscription Expires: 01/01/2020
空闲内存:2147MB
Windows 10 (x64) v10.00, Build 18363

开始设计

打开工程 **新建工程** 导入旧版本文件 打开示例工程

最近打开的工程

- E:\微机原理与接口技术课程\文件\20-21-2\2021版课件\课件程序2021\第2章\ok\案例2-1 简单流水灯.pdsprj
- E:\微机原理与接口技术课程\文件\20-21-2\2021版课件\课件程序2021\第5章\开关+LED数码管-OK.pdsprj
- E:\微机原理与接口技术课程\文件\20-21-2\2021版课件\课件程序\8255-1.pdsprj
- E:\微机原理与接口技术课程\文件\20-21-2\2021版课件\课件程序2021\第5章\开关+LED数码管.pdsprj

最新消息

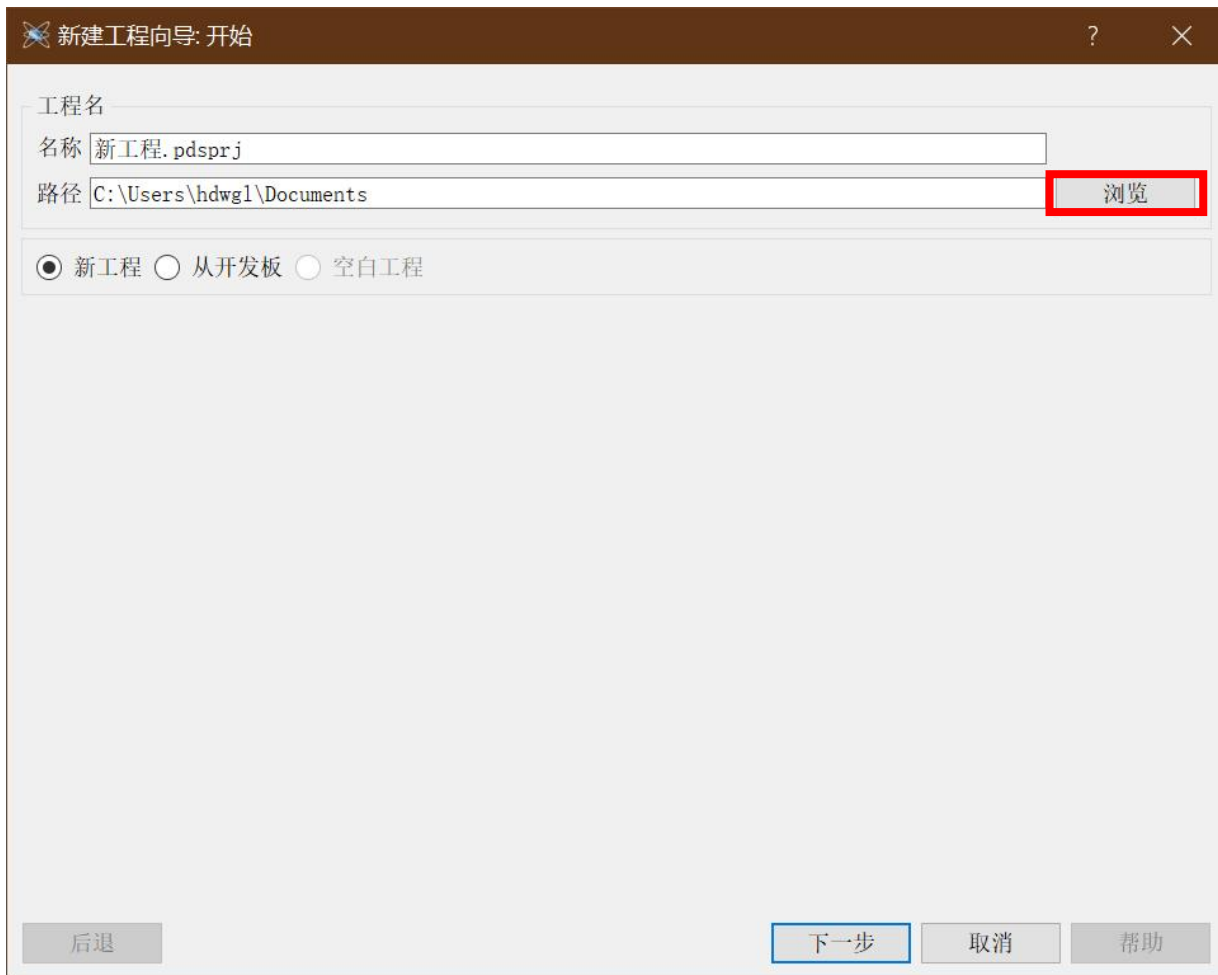
有新版本可升级

⚠ 您的更新许可(USC)已过期。 [更新 USC](#)

描述	发布日期	USC有效	
Proteus Professional 8.12 [8.12.30713]	04/03/2021	否	更新USC
Proteus Professional 8.11 SP1 [8.11.30228]	03/11/2020	否	更新USC
Proteus Professional 8.10 SP3 [8.10.29560]	18/05/2020	是	下载
Proteus Professional 8.9 SP2 [8.9.28501]	05/09/2019	是	下载
Proteus Professional 8.8 SP1 [8.8.27031]	07/11/2018	是	下载
Proteus Professional 8.7 SP3 [8.7.25561]	20/03/2018	是	下载

Windows 任务栏显示时间为 10:55 周三 2021/4/7。

(1) 新建工程



新建工程向导: 开始

工程名

名称 新工程.pdsprj

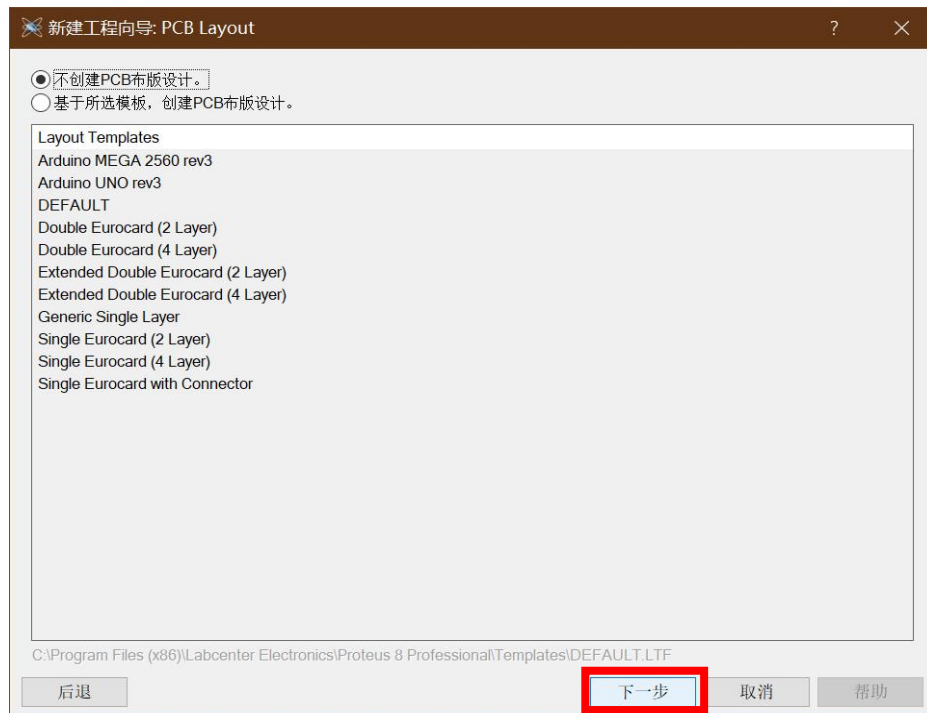
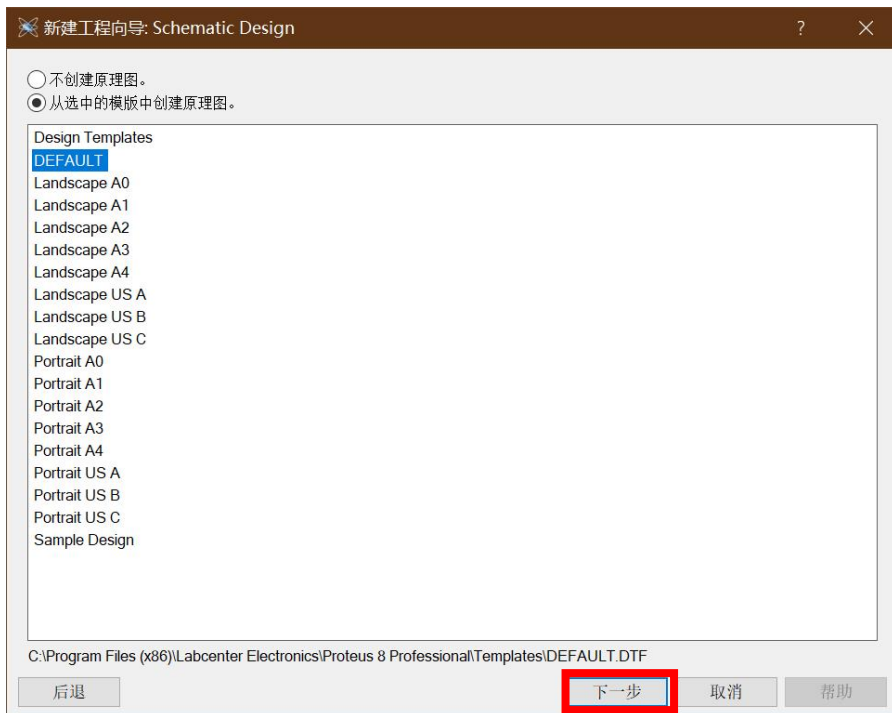
路径 C:\Users\hdwgl\Documents 浏览

☒ 新工程 ☐ 从开发板 ☐ 空白工程

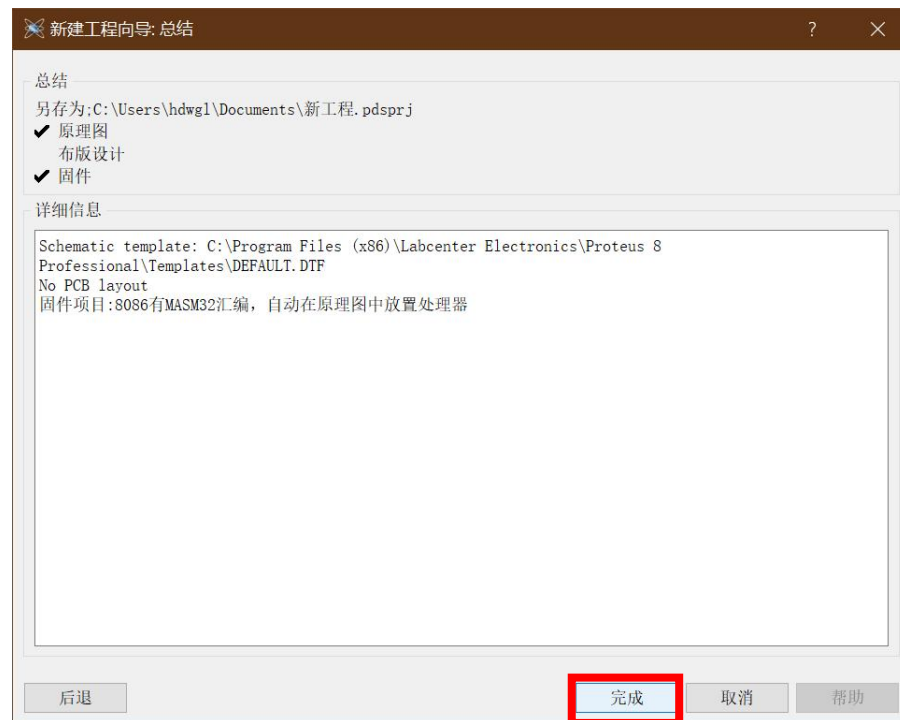
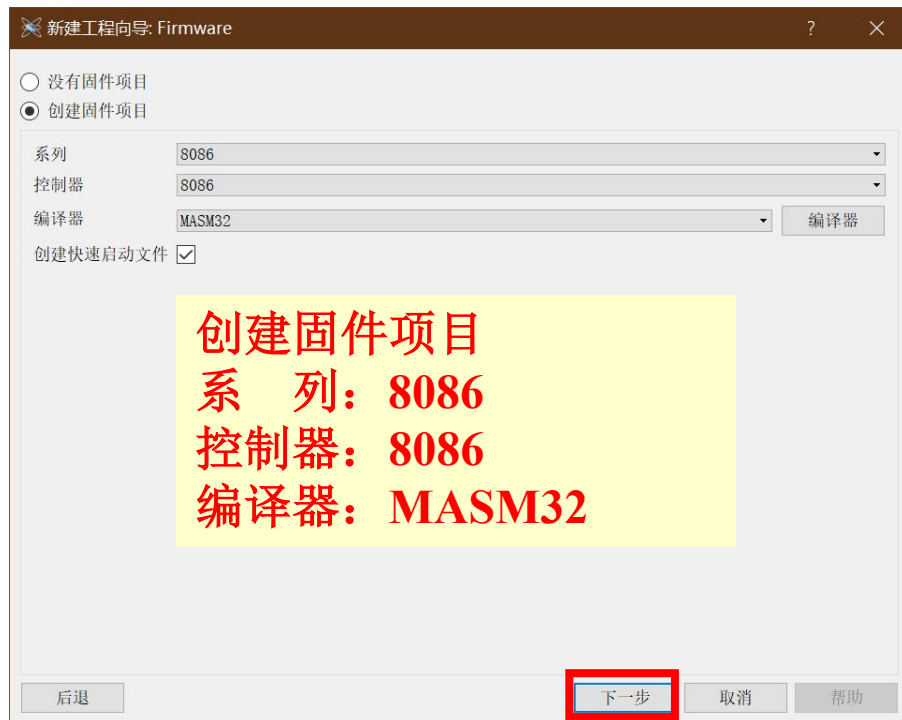
后退 下一步 取消 帮助

选择工程存放路径

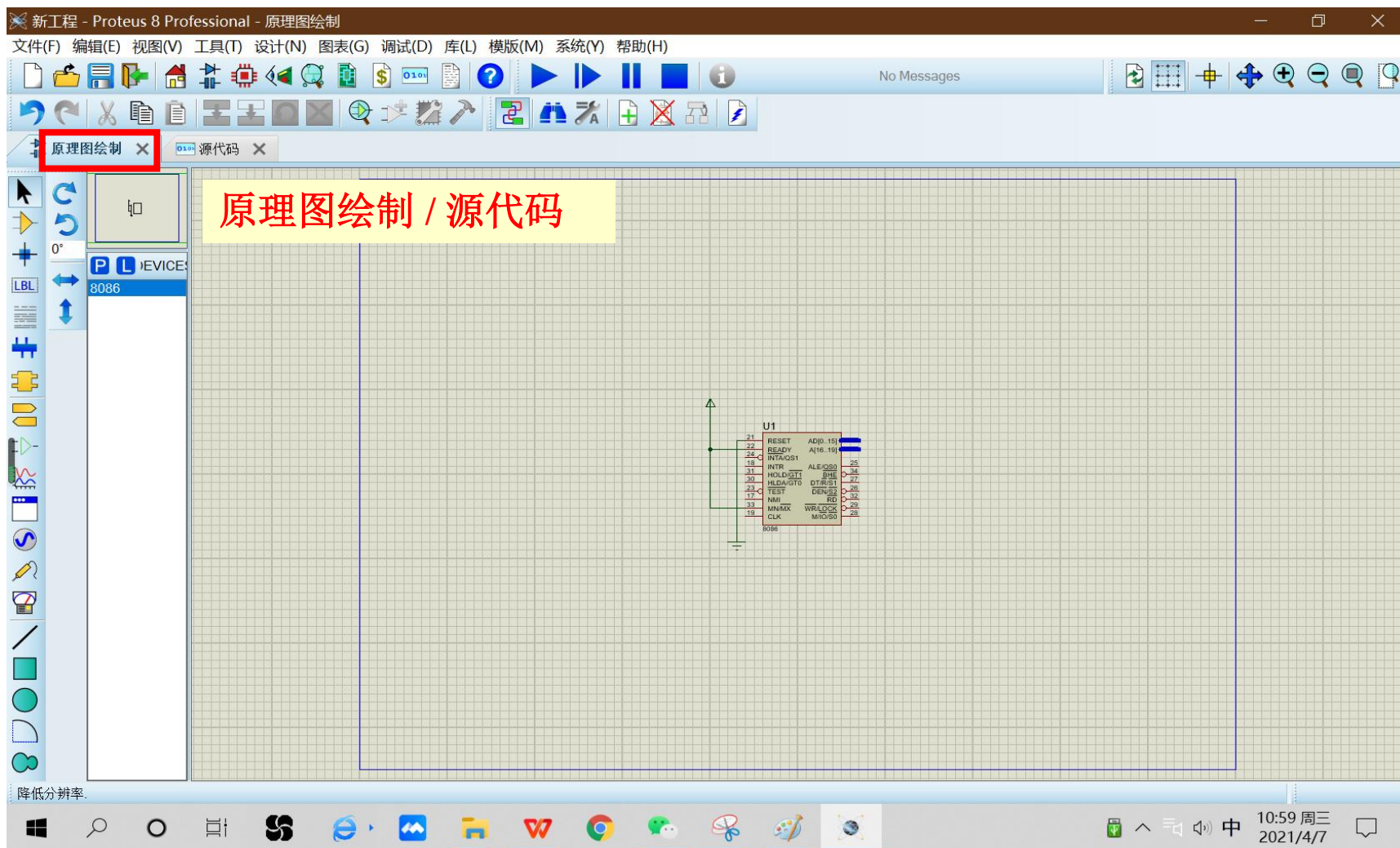
(1) 新建工程



(1) 新建工程



(1) 新建工程



(2) 界面介绍

标题栏

菜单栏

标准工具栏

仿真控制按钮

预览窗口

对

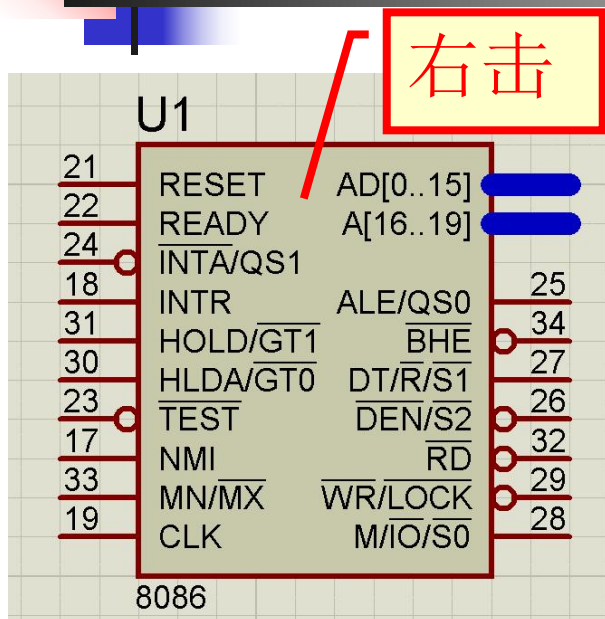
预览

模型

图形编辑窗口

状态栏

(3) 原理图绘制——设置8086内存属性



移动对象
编辑属性 Ctrl+E
删除对象
顺时针旋转 Num--
逆时针旋转 Num++
编辑元件

元件位号(B): U1
元件值(V): 8086
组件(E): 新建(N)

Program File: C:\Users\ADMINI~1\AppData\ Hide All
External Clock: No Hide All
Clock Frequency: 5MHz Hide All
PCB Package: DIL40 Hide All

Advanced Properties:
Internal Memory Size 0x100000 Hide All

Other Properties:

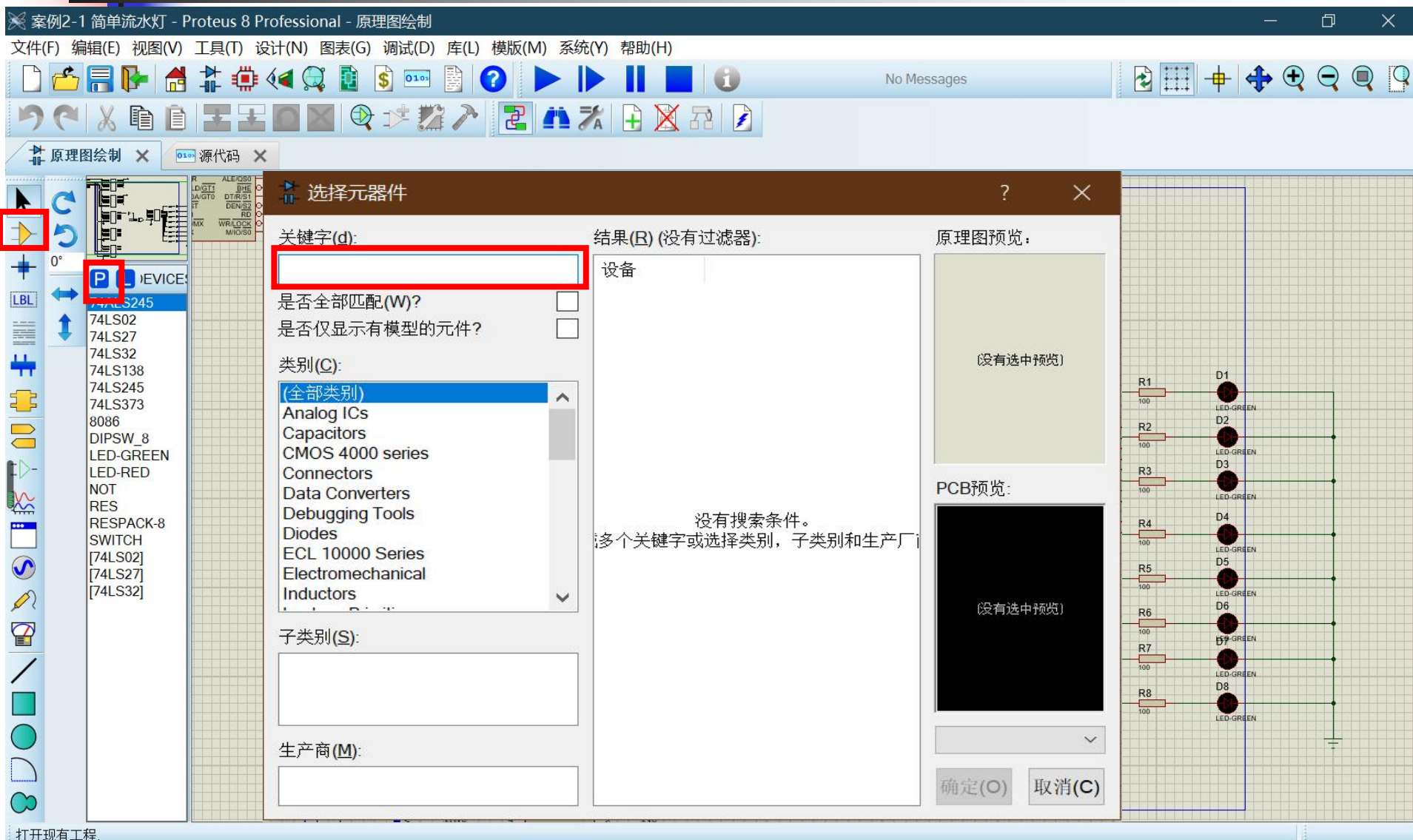
规定8086可使用内存大小
说明：指8086内置存储器

☐ 不进行仿真(S)
☐ 不进行PCB布版(L)
☐ 不放入物料清单(B)

☐ 附加层次模块(M)
☐ 隐藏通用管脚(C)
☐ 使用文本方式编辑所有属性(A)

确定(O)
帮助(H)
数据手册(D)
隐藏引脚(P)
编辑固件(E)
取消(C)

(3) 原理图绘制——选取元件





常用元件的名称

元件名字	元件作用
74LS245	双向缓冲器
74LS373	双向锁存器
74LS02	或非门
74LS32	或门
74LS138	138 译码器
DIPSW_8	8 路开关
LED-RED	红色 led 灯
Resistors-Generic	普通电阻-编辑属性值 100
RESPACK-8	排阻



常用元件的名称

Proteus 中常用元器件名称

一，电阻 Resistors 和单片机 Microprocessor Ics AT89C51

排阻 R*8 上拉电阻 RESPACK8 可变电阻 POT

二，开关和继电器 Switchs&Relays

BUTTON 按钮开关 Switch 开关 TSC-112D3H 继电器

DIPSW_8 拨动开关 SW-SPST 单刀单掷开关 SW-SPDT 单

刀双掷开关

W107DIP-4 固态继电器 SW-ROT-6 单刀 6 掷开关

常用元件的名称

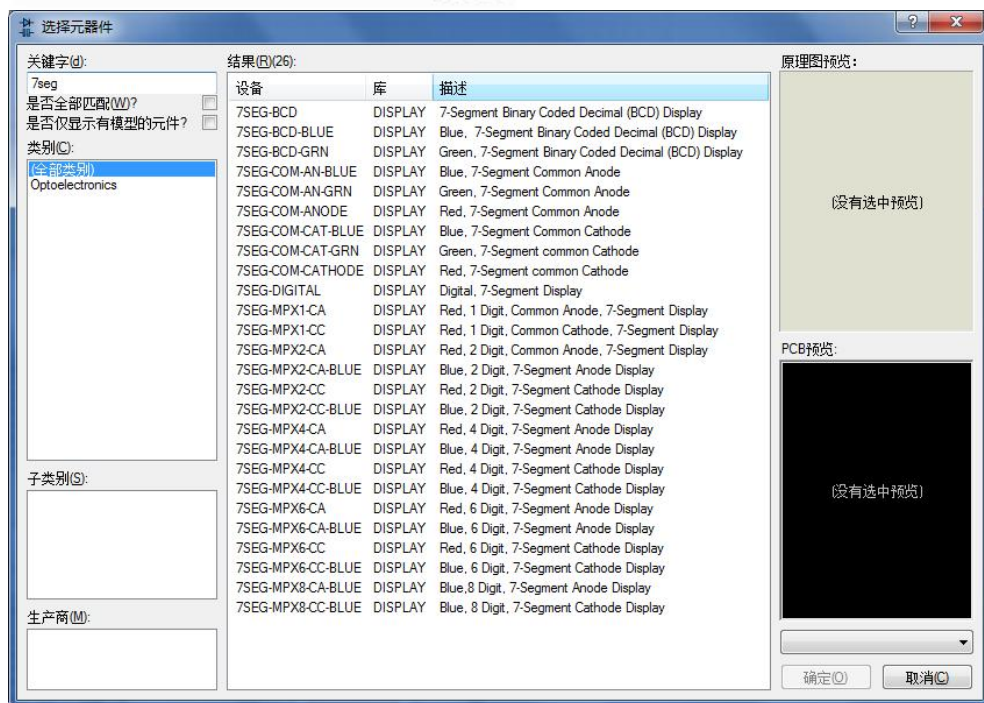
四,光电器件 Optoelectronics

LAMP 灯泡 LED-BIBY 高亮度黄灯 LED LED-GREEN 绿色发光二极管

LED-BIRG 高亮度绿灯 LED LED-RED 红色发光二极管

LED-BIRG 高亮度蓝灯 LED LED-YELLOW 黄色发光二极管

LED-BARGRAPH-RED 光柱



7SEG-COM-ANODE: 共阳极数码管、
7SEG-COM-CATHODE: 共阴极数码管。

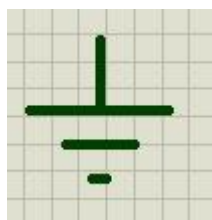
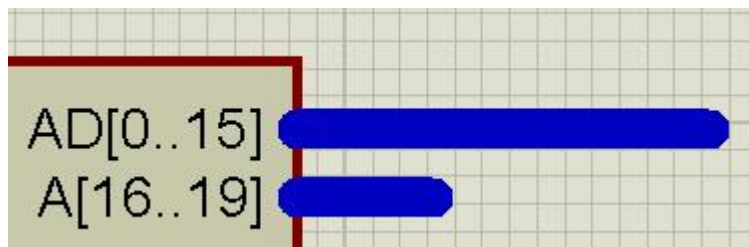
(3) 原理图绘制-- 整体布局与连线

◆ 疏落有致，美观



-----总线模式

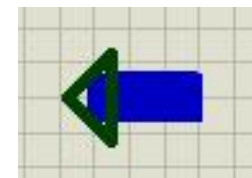
-----终端模式



GROND
-接地，



POWER
-电源，



BUS
-总线终端

(4) 编写源程序

新工程 - Proteus 8 Professional - 源代码

文件(F) 源码(S) 构建(B) 编辑(E) 调试(D) 系统(Y) 帮助(H)

原理图绘制 源代码

工程

- 8086 (U1)
- Source 文件
 - main.asm

```
1 ;=====
2 ; Main.asm file generated by New Project wizard
3 ;
4 ; Created:   周三 4月 7 2021
5 ; Processor: 8086
6 ; Compiler:  MASM32
7 ;
8 ; Before starting simulation set Internal Memory Size
9 ; in the 8086 model properties to 0x10000
10 ;=====
11
12 CODE    SEGMENT PUBLIC 'CODE'
13         ASSUME CS:CODE
14
15 START:
16         ; Write your code here
17 ENDLESS:
18         JMP ENDLESS
19 CODE    ENDS
20 END START
```

VSM Studio输出

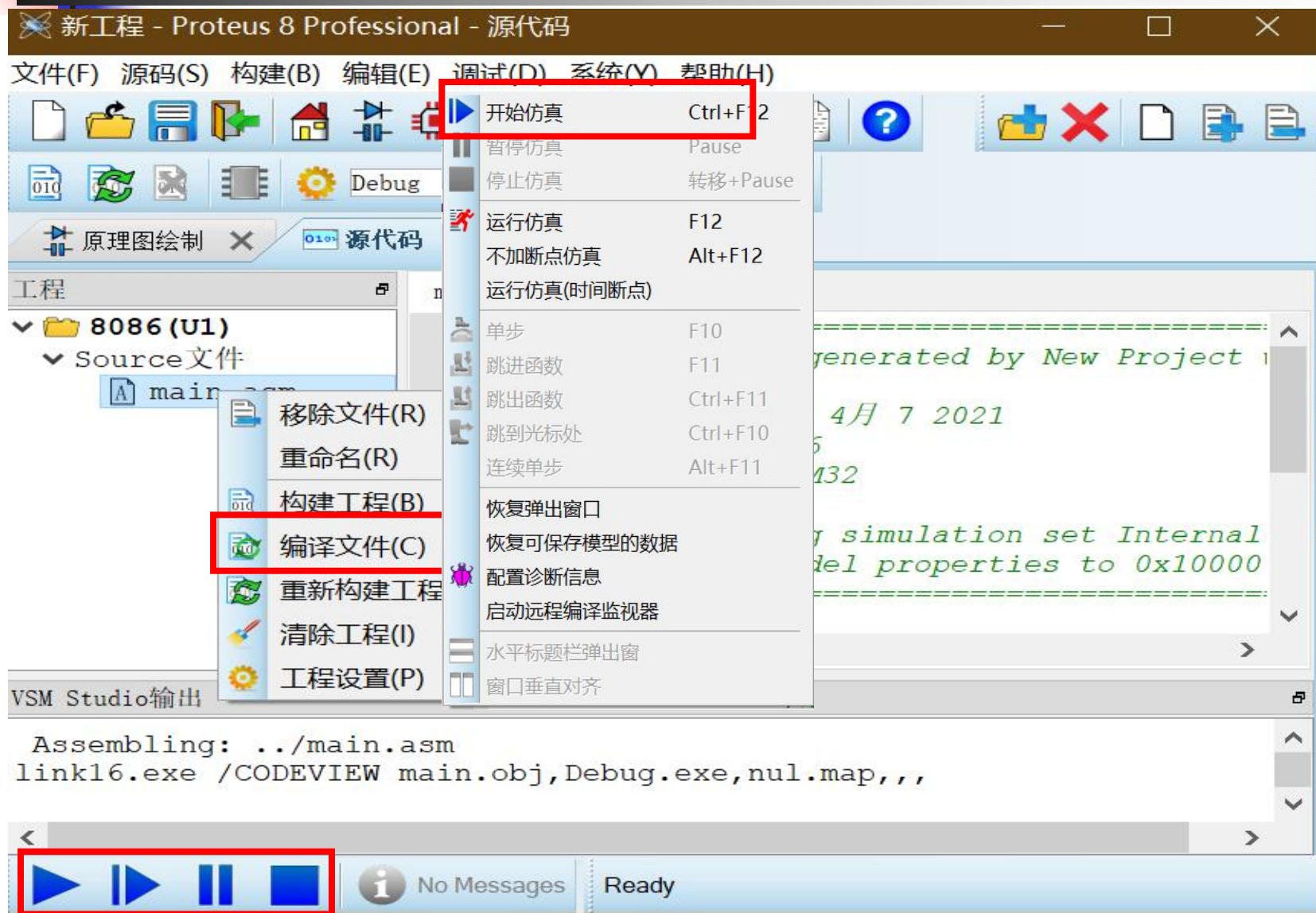
```
Assembling: ../main.asm
link16.exe /CODEVIEW main.obj,Debug.exe,nul.map,,,

Microsoft (R) Segmented Executable Linker  Version 5.60.339 Dec  5 1994
Copyright (C) Microsoft Corp 1984-1993.  All rights reserved.

LINK : warning L4021: no stack segment
编译成功。
```

14:47 周三 2021/4/7

(5) 编译、运行程序



(5) 编译、运行程序

案例2-1 简单流水灯 - Proteus 8 Professional - 源代码

文件(F) 源码(S) 构建(B) 编辑(E) 调试(D) 系统(Y) 帮助(H)



原理图绘制 x 源代码 x

8086 Source Code - U1

main.asm

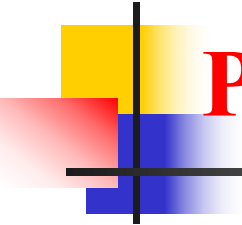
```
----- Main.asm file generated by New Project wizard -----
Created:  周四 三月 12 2020
Processor: 8086
Compiler:  MASM32
-----
Before starting simulation set Internal Memory Size
in the 8086 model properties to 0x10000
-----
CODE      SEGMENT PUBLIC 'CODE'
ASSUME    CS:CODE
START:
; Write your code here
0000      MOV BL,00000001B ;设置流水灯初始控制码, 点亮第1个灯
0002      ENDLESS:MOV AL,BL ;若不为0, 则开关为打开状态, 输出流水灯控制码BL的值
0004      OUT 02H,AL
0006      ROL BL,01H ;流水灯控制码左循环1位, 即下一个灯亮, 其余灯灭
0008      CALL DELAY100 ;调用延时子程序,软件延时约1秒
000B      JMP ENDLESS
----- DELAY100 PROC ;延时子程序, CPU执行下面程序耗时约1s
000D      PUSH CX
000E      MOV CX,0
0011      LOOP $
0013      LOOP $
0015      LOOP $
0017      MOV CX,15000
001A      LOOP $
001C      POP CX
001D      RET
----- DELAY100 ENDP
----- CODE      ENDS
----- END START
```

	运行仿真	F12
	单步	F10
	跳进函数	F11
	跳出函数	Ctrl+F11
	跳到光标处	Ctrl+F10
	连续单步	Alt+F11

8086 Variables - U1

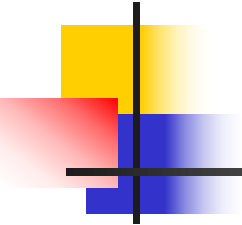
Name	Address	Value

7 Message(s)



Proteus仿真练习

1. 完成Proteus8.4的安装;
2. 完成案例2-1和2-2, 将仿真文件(后缀名为pdsprj)上传到课堂派。



第2章作业

P38

2.1 、 2.2、 2.3 、 2.4、

2.5、 2.6、 2.10、